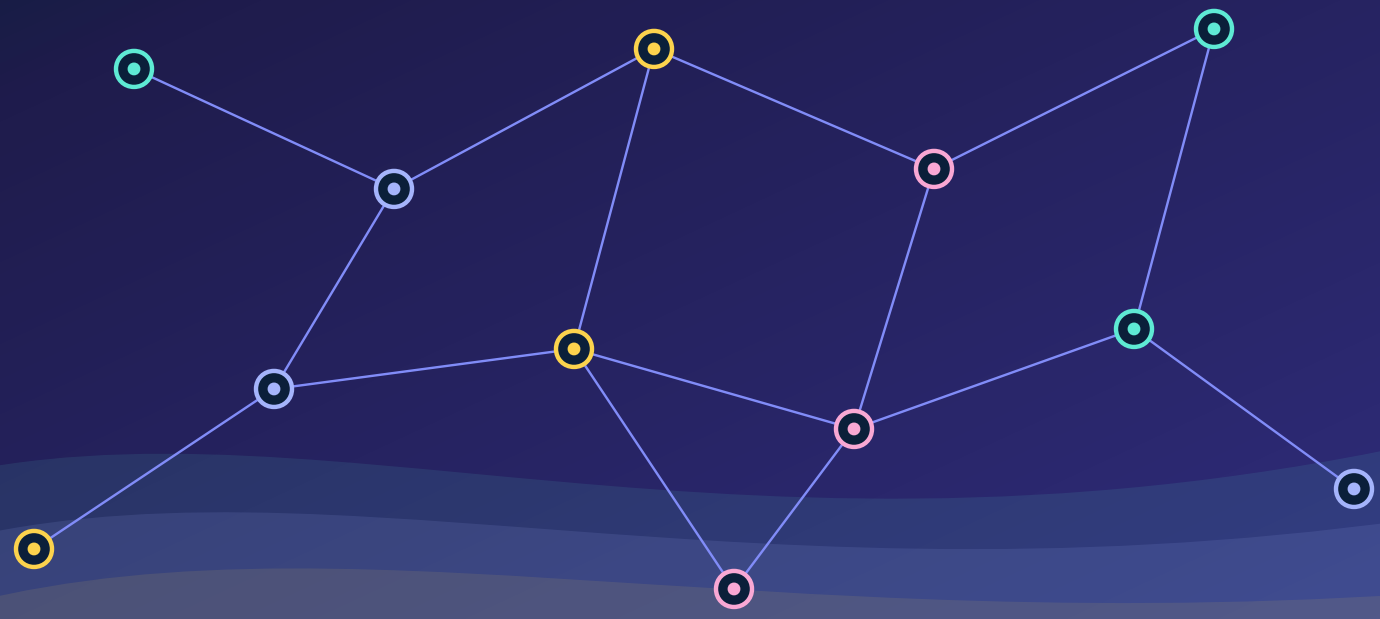
INTERVIEW HANDBOOK · 2026 EDITION

Data Engineering Terms Explained

From ACID and the Write-Ahead Log to lakehouses, CDC, Delta Live Tables and dbt: every concept you need for the interview, with diagrams.

48 topics · 50+ diagrams · Interview Q&A · Glossary



Contents

Frequency stars show how often a topic comes up in data engineering interviews.

PART 1 — DATABASE FOUNDATIONS

1.1 CRUD Operations 4
1.2 ACID Properties 5
1.3 Atomicity (All-or-Nothing) 6
1.4 Write-Ahead Log (WAL) & WAL Buffer 7
1.5 Isolation Levels & MVCC 8
1.6 Indexes (B-Tree) 9
1.7 OLTP vs OLAP (Row vs Column Storage) 10
1.8 Normalization (1NF → 3NF) 11
1.9 Database Modeling & Data Modeling 12
1.10 Dimensional Modeling: Star & Snowflake Schema 13
1.11 Slowly Changing Dimensions (SCD) 15

PART 2 — STORAGE, PLATFORMS & ARCHITECTURE

2.1 Data Warehouse 18
2.2 Data Lake 19
2.3 Data Lakehouse 20
2.4 Parquet (Columnar File Format) 21
2.5 Open Table Formats, Delta Lake & the Transaction Log 22
2.6 Apache Spark 24
2.7 Trino 26
2.8 Lakebase 27
2.9 Data Mart 28
2.10 Data Mesh 29
2.11 Data Fabric 30
2.12 Medallion Architecture (Bronze, Silver, Gold) 31

PART 3 — PIPELINES, PROCESSING & DATAOPS

3.1 Data Pipelines: ETL vs ELT 34
3.2 Incremental Data Loading 35
3.3 Watermarks 36
3.4 Back-date Refresh (Lookback & Backfill) 37
3.5 Idempotency 38
3.6 Exactly-Once Processing 39
3.7 Upsert (MERGE) 40
3.8 Batch vs Streaming Data Processing 41
3.9 Lambda vs Kappa Architecture 42
3.10 Change Data Capture (CDC) 43
3.11 Delta Live Tables (Lakeflow Declarative Pipelines) 45
3.12 dbt (Data Build Tool) 47
3.13 CI/CD & DataOps 48

PART 4 — OPERATING AT SCALE & INTERVIEW TOOLKIT

4.1 Partitioning, Clustering & the Small-File Problem 51
4.2 Shuffle, Skew & Join Strategies 52
4.3 Schema Enforcement, Evolution & Data Contracts 53
4.4 Data Quality, Observability, Lineage & Governance 54
4.5 Orchestration (Airflow & DAGs) 55
4.6 Kafka & Message Brokers 56
4.7 CAP Theorem, Replication & Sharding 57
4.8 Data Vault, Surrogate Keys & Modeling Choices 58
4.9 Performance Tuning Cheat Sheet 59
4.10 How to Answer a Pipeline Design Question 60
4.11 Rapid-Fire Interview Q&A 61
4.12 Glossary: Every Term in One Line 63

How to use this handbook

Read the **In one line** box first, study the diagram, then skim the **Interview angle** callouts. Finish with Part 4: the design template, Q&A and glossary.

Beyond your list

Added topics: indexes, isolation levels, row vs column storage, partitioning, small files, shuffle & skew, schema evolution, data contracts, quality & governance, Airflow, Kafka, CAP, Data Vault, and performance tuning.

01

Part 1 · Database Foundations

How databases keep data correct and fast, and how we model it for transactions and analytics.

- CRUD & ACID
- Atomicity & WAL / WAL Buffer
- Isolation levels & Indexes
- OLTP vs OLAP
- Normalization
- Database & Data Modeling
- Star & Snowflake Schema
- Slowly Changing Dimensions

1.1 CRUD Operations

★★☆

IN ONE LINE

Create, Read, Update, Delete — the four basic operations performed on persistent data.

CRUD is the vocabulary of every operational database and API. Data engineers care because **the same four verbs behave very differently** in an OLTP database (cheap, in-place, row-level) versus an analytical lake (immutable files, expensive rewrites).

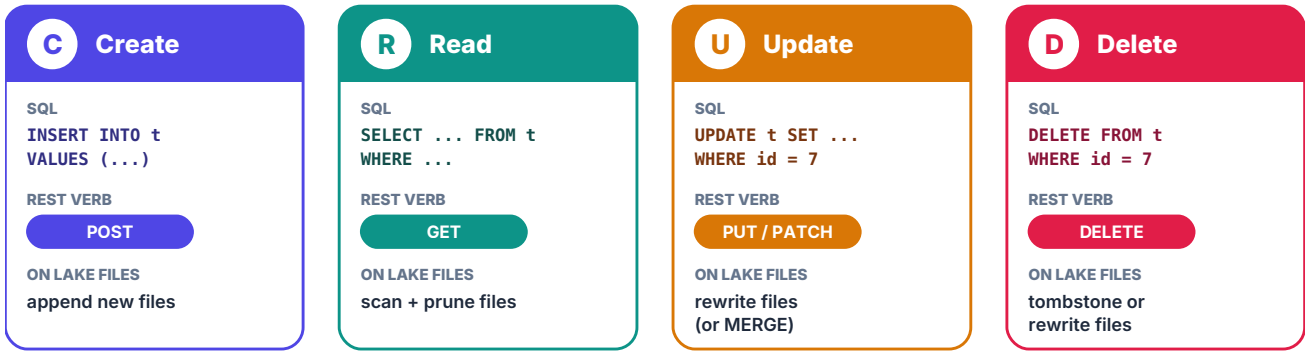


Figure 1.1 — CRUD across SQL, REST and file-based lakes

Op	SQL	Cost in OLTP DB	Cost on a data lake
Create	`INSERT`	Cheap — one row, one index update	Cheapest — just append a new Parquet file
Read	`SELECT`	Fast via index lookups	Fast if partitioned/clustered; scans otherwise
Update	`UPDATE`	Cheap — in-place (MVCC new version)	Costly — files are immutable, so rewrite the file (copy-on-write)
Delete	`DELETE`	Cheap — mark tuple dead, vacuum later	Costly — rewrite files, or write a *deletion vector*

INTERVIEW ANGLE

"Why are UPDATE and DELETE hard on a data lake?" Parquet files are immutable. Changing one row means rewriting the whole file that contains it. Table formats (Delta, Iceberg, Hudi) hide this behind transactions, and add tricks like **deletion vectors** / merge-on-read so you do not rewrite a 500 MB file to remove one row.

1.2 ACID Properties

★★★

IN ONE LINE

Atomicity, Consistency, Isolation, Durability — the four guarantees that make database transactions trustworthy.

ACID is the contract a relational database makes about **transactions** — a group of operations treated as one logical unit. It is the single most-asked database concept in interviews, usually followed by "how does the DB actually implement it?"

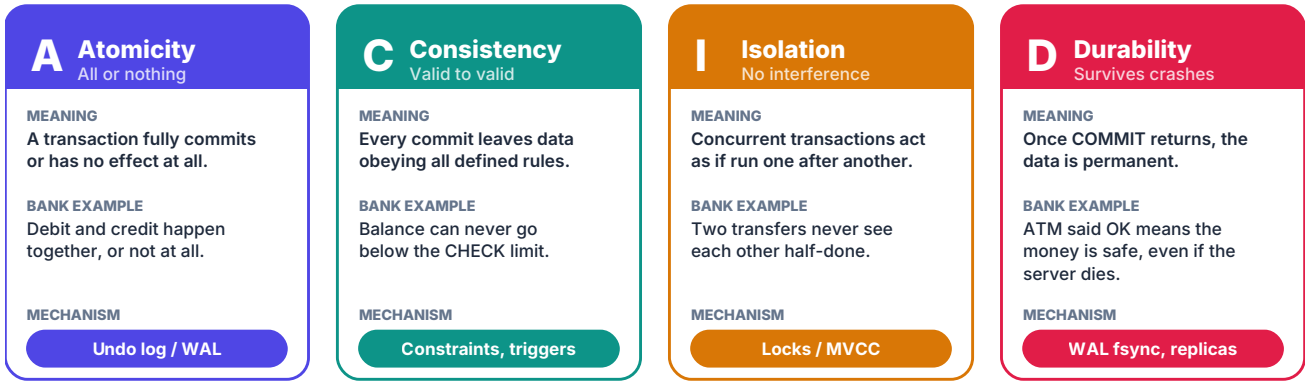


Figure 1.2 — The four ACID guarantees and the mechanism behind each

ACID → SQL / OLTP world

Strong guarantees, strict correctness. PostgreSQL, MySQL (InnoDB), Oracle, SQL Server. Also delivered on the lake by Delta Lake, Iceberg and Hudi.

BASE → many NoSQL systems

Basically **A**vailable, **S**oft state, **E**ventually consistent. Trades strictness for availability and horizontal scale (e.g. early Cassandra, DynamoDB eventual reads).

COMMON TRAP

The "C" in ACID (data obeys *application rules*) is **not** the "C" in the CAP theorem (all nodes see the same data). Interviewers love this mix-up. Also: consistency in ACID is partly the *application's* job — the DB only enforces rules you declared.

1.3 Atomicity (All-or-Nothing)

★★★

IN ONE LINE

A transaction either completes entirely or has no effect whatsoever.

Atomicity means a transaction is **indivisible**. If any step fails — constraint violation, crash, network drop, deadlock victim — the database undoes every change made so far so nobody ever observes a half-finished state.

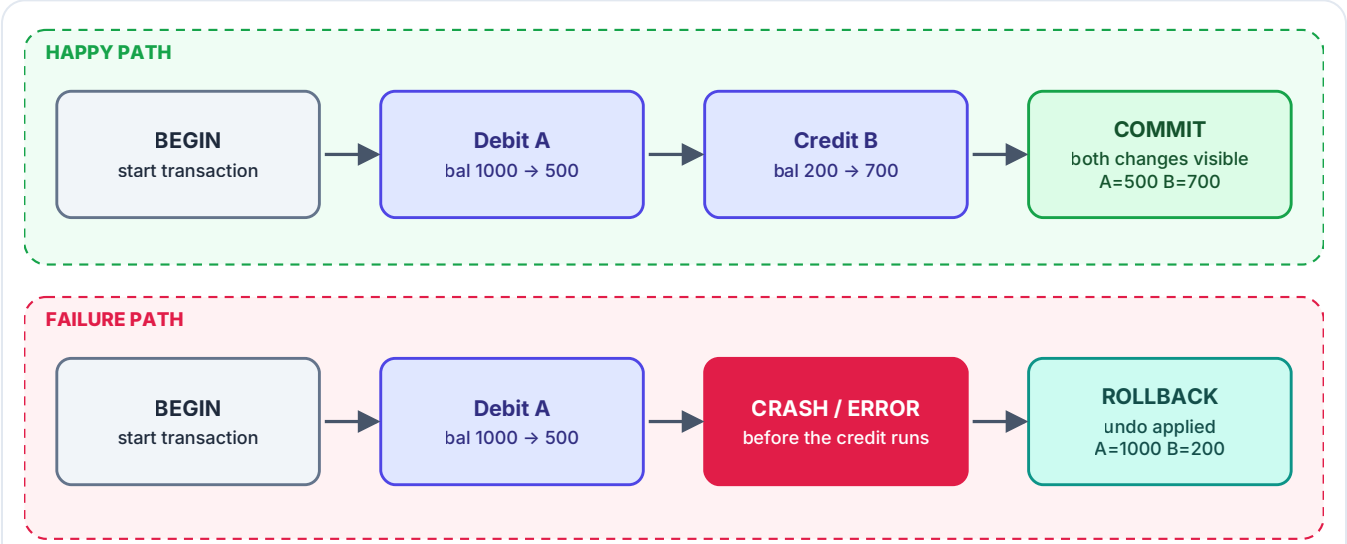


Figure 1.3 — Transfer 500 from account A to B: commit vs rollback

How is it implemented?

- **Undo information** — before changing a row the DB records enough to reverse it (undo log in MySQL/Oracle, old tuple versions in PostgreSQL MVCC).
- **Write-Ahead Log (next topic)** — every change is logged first. On crash recovery, uncommitted work is rolled back and committed work is replayed.
- **Commit as a single atomic step** — a transaction becomes visible only when its COMMIT record is durably in the log.

SQL · TRANSACTION

```sql
BEGIN;
  UPDATE accounts SET balance = balance - 500 WHERE id = 'A';
  UPDATE accounts SET balance = balance + 500 WHERE id = 'B';
COMMIT;   -- or ROLLBACK; to cancel everything
```

REMEMBER

Atomicity also shows up on the lake: a Delta Lake `MERGE` that fails halfway leaves **no partial files visible** because readers only trust files listed in the committed transaction log.

1.4 Write-Ahead Log (WAL) & WAL Buffer

★★★

IN ONE LINE

Changes are written to an append-only log *before* data files, so a crash can always be recovered or rolled back.

The core rule of WAL is simple: **log the change before you apply it to the data files**. Random writes to data pages are slow; sequentially appending a small log record is fast. So the database only has to make the *log* durable at commit time — the data pages can be flushed whenever convenient.

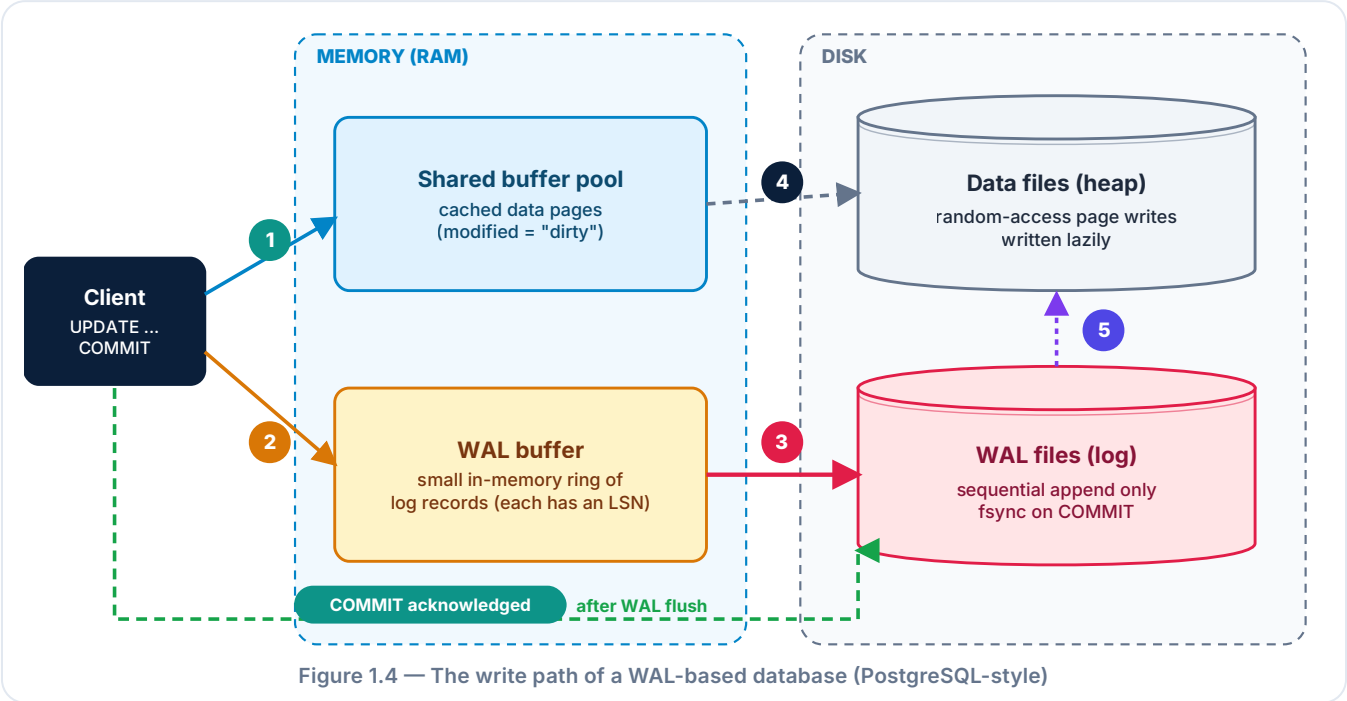


Figure 1.4 — The write path of a WAL-based database (PostgreSQL-style)

The five steps

1. Modify the page in the **shared buffer pool** (RAM) → it becomes a *dirty page*.
2. Append a log record describing the change to the **WAL buffer**; each record gets a Log Sequence Number (**LSN**).
3. On `COMMIT`, flush the WAL buffer to the **WAL file** and `fsync`. Only now is the client told "OK".
4. A **checkpoint** / background writer flushes dirty pages to data files later.
5. After a crash, **replay the WAL** from the last checkpoint (REDO) and discard uncommitted work.

Why it matters

- **Durability** — committed data survives power loss.
- **Atomicity** — uncommitted changes can be undone.
- **Performance** — sequential log writes, *group commit* batches many transactions into one fsync.
- **Replication** — replicas just stream and replay the WAL.
- **CDC** — tools like Debezium read the WAL (PostgreSQL) or binlog (MySQL) to capture changes with no load on tables.

REMEMBER

WAL buffer = the in-memory staging area for log records, flushed to disk at commit (or when full). Tuning it (e.g. `wal_buffers` in PostgreSQL) mainly helps write-heavy workloads with many concurrent commits.

INTERVIEW ANGLE

The same idea powers **Delta Lake's transaction log**, Kafka's commit log and LSM-tree databases. If asked "what is common between a database and Kafka?" — *both are built around an append-only log*.

1.5 Isolation Levels & MVCC

★★☆

IN ONE LINE

How much concurrent transactions are allowed to see of each other's in-flight work.

Perfect isolation (serializable) is expensive, so databases let you pick a weaker level and accept specific anomalies in exchange for concurrency.

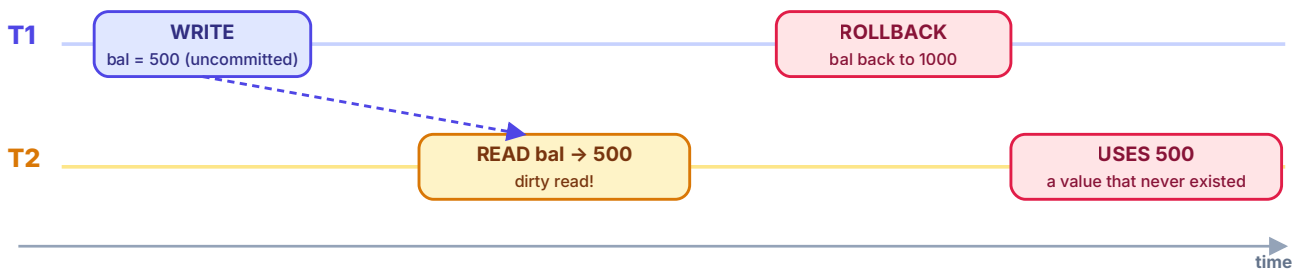


Figure 1.5 — A dirty read: T2 acts on data T1 later rolls back

Isolation level	Dirty read	Non-repeatable read	Phantom read	Typical default
Read Uncommitted	possible	possible	possible	rarely used
Read Committed	**prevented**	possible	possible	PostgreSQL, Oracle, SQL Server
Repeatable Read	prevented	**prevented**	possible*	MySQL InnoDB
Serializable	prevented	prevented	**prevented**	strictest, slowest

Anomalies

- **Dirty read** — reading uncommitted data.
- **Non-repeatable read** — same row read twice returns different values.
- **Phantom read** — same query returns new/missing *rows*.

How it's implemented

- **Locks** (2PL) — readers can block writers.
- **MVCC** — each transaction reads a snapshot; readers never block writers. Used by PostgreSQL, Oracle, InnoDB, and Delta Lake (snapshot isolation).

* PostgreSQL's Repeatable Read also prevents phantoms (snapshot isolation); the SQL standard only requires it at Serializable.

1.6 Indexes (B-Tree)

★★☆

IN ONE LINE

A sorted lookup structure that avoids full table scans at the cost of extra storage and slower writes.

An index is a separate, sorted structure that lets the database jump to rows instead of scanning the whole table. The default is the **B-tree** (balanced, sorted, great for equality and range lookups). Lookup cost is O(log n).

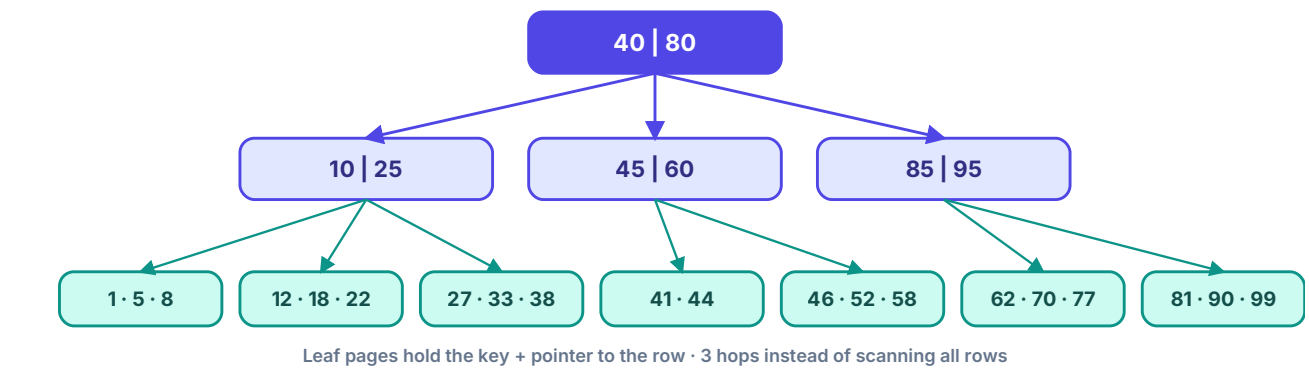


Figure 1.6 — B-tree: root, internal and leaf pages

Index type	Best for	Note
B-tree	equality, ranges, ORDER BY	Default almost everywhere
Hash	equality only	No range scans
Composite	multi-column filters	Column order matters (leftmost-prefix rule)
Covering	index-only scans	Index contains all needed columns
Bitmap / GIN	low-cardinality, arrays, text	Analytical / full-text use

COMMON TRAP

Indexes speed reads but **slow every write** (each INSERT/UPDATE must also update indexes) and consume storage. Column stores and lakehouses usually skip B-trees entirely and rely on **partition pruning, min/max statistics and clustering** instead.

1.7 OLTP vs OLAP (Row vs Column Storage)

★★★

IN ONE LINE

OLTP processes many small transactions; OLAP runs complex analytical queries over large historical data.

The two workload families that shape every data architecture. **OLTP** runs the business (many small, fast, concurrent transactions). **OLAP** analyses it (fewer, heavy, read-mostly queries scanning huge volumes). Their conflicting needs are why we copy data from one into the other.

	OLTP	OLAP
Purpose	Run day-to-day operations	Analytics, reporting, ML
Typical query	Insert/update one order; look up one customer	Revenue by region by month for 5 years
Rows touched	A handful	Millions to billions
Latency target	Milliseconds	Seconds to minutes
Schema	Normalised (3NF)	Denormalised (star / wide tables)
Storage layout	**Row-oriented**	**Column-oriented**
Concurrency	Thousands of small transactions	Tens of big queries
Examples	PostgreSQL, MySQL, Oracle, DynamoDB, Lakebase	Snowflake, BigQuery, Redshift, Databricks SQL, Trino

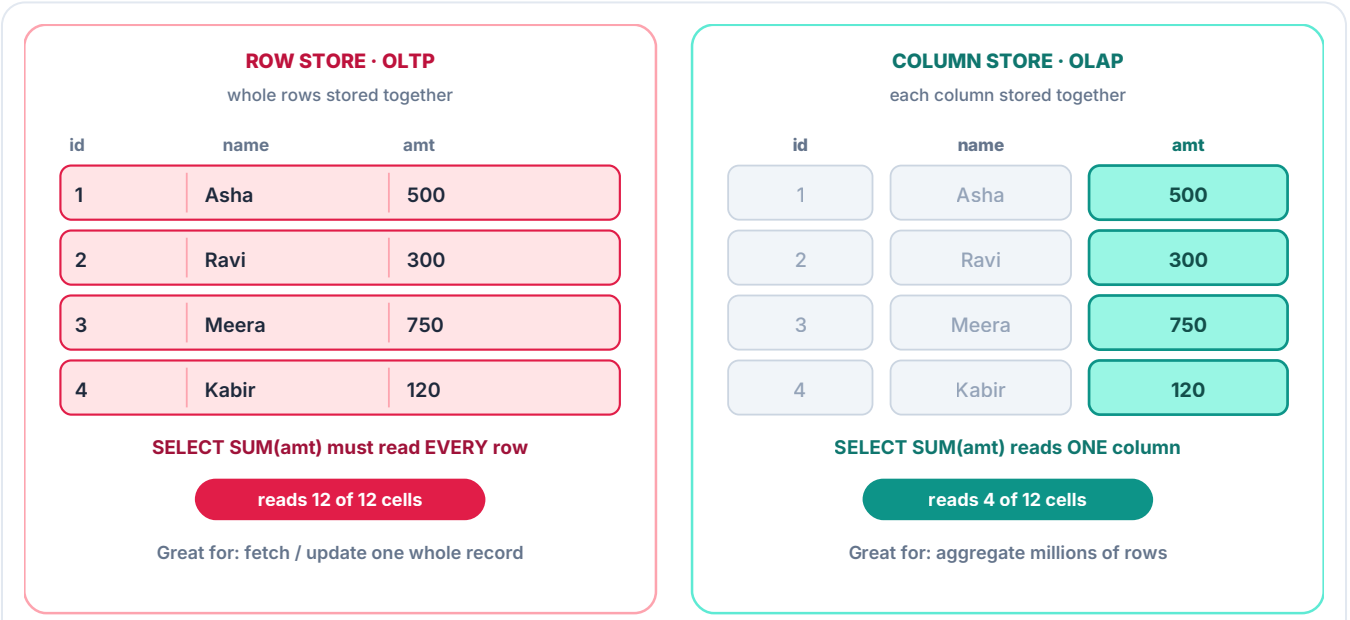


Figure 1.7 — Row vs column layout: why analytics engines read far less data

INTERVIEW ANGLE

Columnar layout also compresses much better (similar values sit together), enables **vectorised execution** and is the reason Parquet/ORC dominate analytics. A hybrid, **HTAP**, tries to serve both workloads in one system (e.g. TiDB, SingleStore).

1.8 Normalization (1NF → 3NF)

★★★

IN ONE LINE

Organising tables to remove redundancy and update anomalies by storing each fact once.

Normalisation removes redundancy by splitting data into tables so each fact lives in exactly **one place**. It prevents update, insert and delete anomalies and is the standard for OLTP schemas.

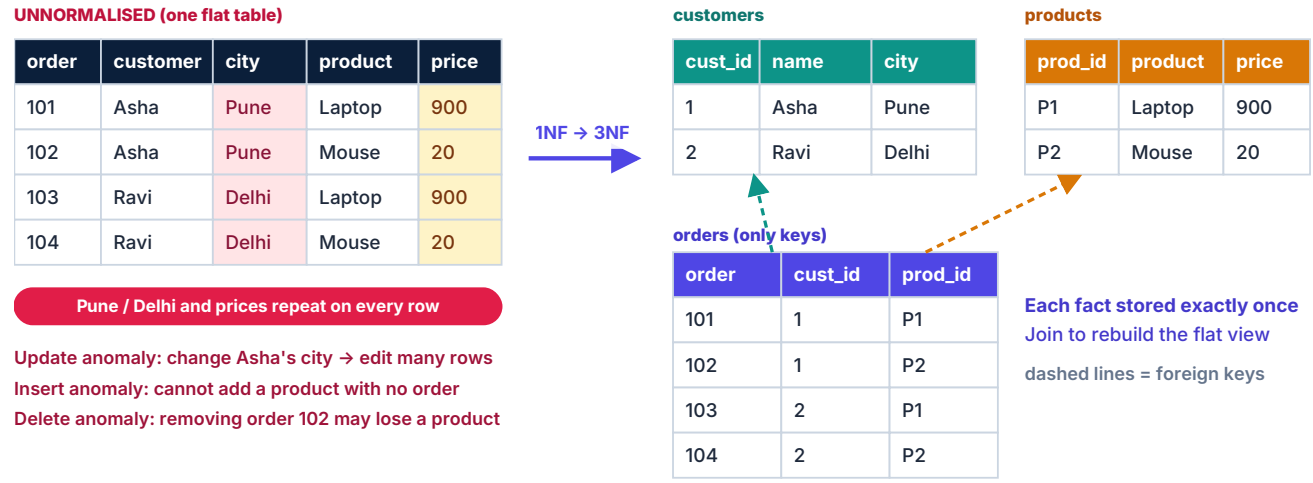


Figure 1.8 — From one repetitive table to three clean ones joined by keys

Form	Rule	Fix
1NF	Every cell is atomic; no repeating groups or arrays; each row unique	Split *"phones: 555,666"* into separate rows
2NF	1NF + no **partial** dependency (non-key column depends on only part of a composite key)	Move the partially-dependent columns to their own table
3NF	2NF + no **transitive** dependency (non-key column depends on another non-key column)	city → zip is a separate table, not on the customer
BCNF	Every determinant is a candidate key (stricter 3NF)	Rare edge cases with overlapping keys

INTERVIEW ANGLE

Memory hook: every non-key attribute must depend on **"the key, the whole key, and nothing but the key — so help me Codd."** Then add the twist: *analytics deliberately denormalises* (star schema, One Big Table) because joins are costly at scale and data is read-mostly.

1.9 Database Modeling & Data Modeling

★★★

IN ONE LINE

Designing how data is structured — from business concepts down to physical tables.

Database modelling usually means designing the schema of an operational (OLTP) system — ER diagrams, normalised tables, constraints. **Data modelling** is the wider discipline: shaping data for *any* purpose, including analytics (dimensional, Data Vault, wide tables). Both are done at three levels of detail.

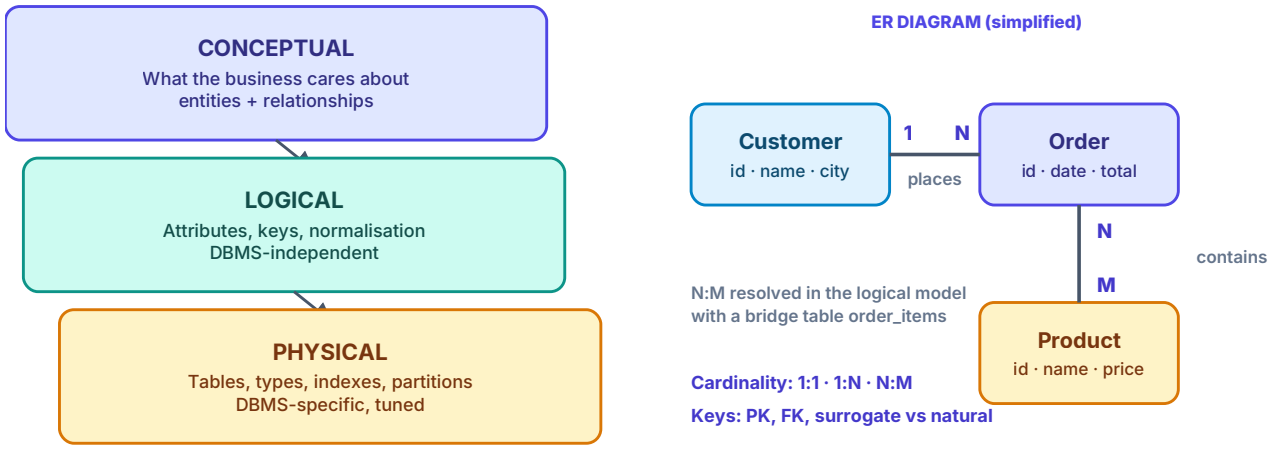


Figure 1.9 — Conceptual → logical → physical modelling, and a simple ER diagram

Approach	Where used	Shape	Strength
ER / 3NF	OLTP, Inmon-style EDW core	Many small normalised tables	Integrity, no redundancy
Dimensional (Kimball)	BI, marts, Gold layer	Fact + dimension tables	Fast, intuitive queries
Data Vault	Enterprise integration, auditability	Hubs, Links, Satellites	Agile, historised, parallel loads
One Big Table (OBT)	ML features, dashboards	One denormalised wide table	No joins at query time

INTERVIEW ANGLE

When asked "how would you model X?", clarify the **access pattern** first. OLTP → normalise. Analytics → pick the **grain**, then facts and dimensions. Naming the grain out loud is the single biggest signal of a senior modeller.

1.10 Dimensional Modeling: Star & Snowflake Schema

★★★

IN ONE LINE

Facts (measurements) surrounded by dimensions (context), designed around the business-process grain.

Dimensional modelling (Ralph Kimball) organises analytic data into **fact tables** (measurable events: sales, clicks, shipments) surrounded by **dimension tables** (descriptive context: who, what, where, when). The first design decision is always the **grain** — exactly what one row of the fact table represents.

Star schema

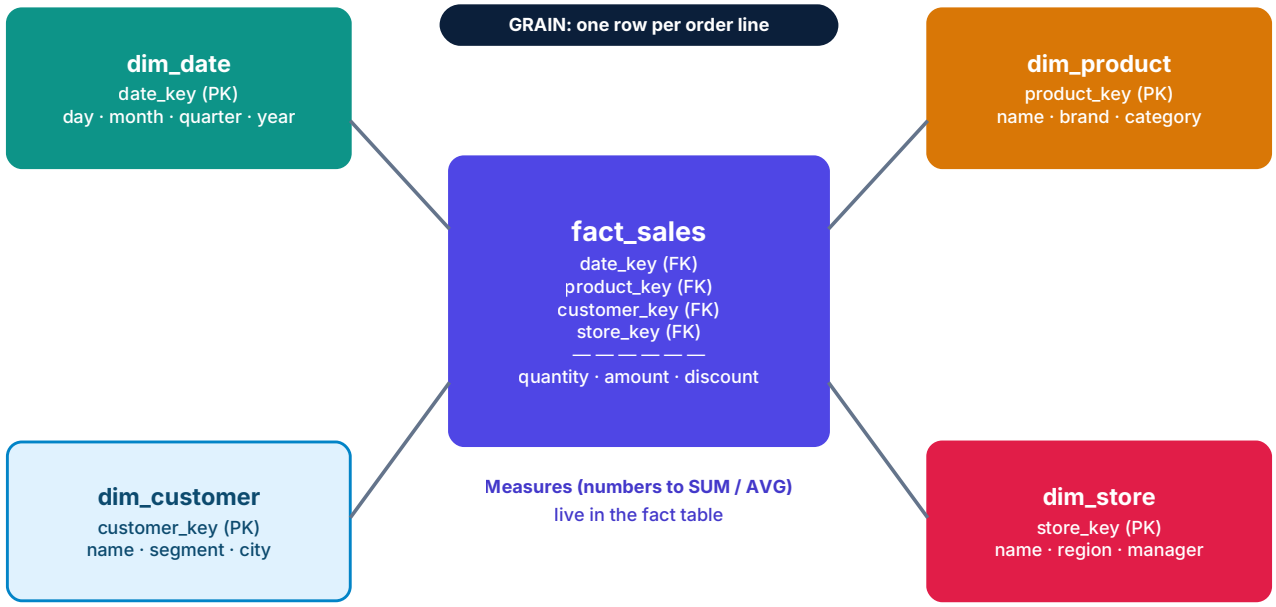


Figure 1.10a — Star schema: one fact table, denormalised dimensions one join away

Snowflake schema

Same idea, but dimensions are **further normalised** into sub-dimensions.

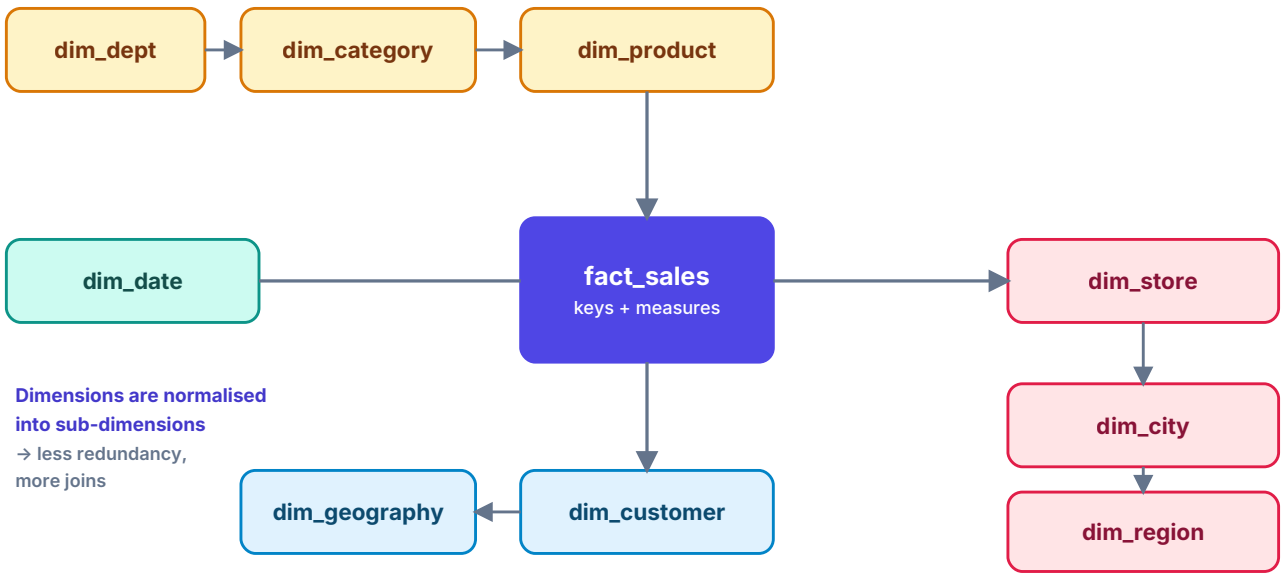


Figure 1.10b — Snowflake schema: dimensions split into hierarchies

	Star schema	Snowflake schema
Dimensions	Denormalised (flat)	Normalised (hierarchy of tables)
Joins per query	Fewer → faster	More → slower, more complex
Storage / redundancy	More redundancy	Less redundancy
Maintenance	Simple ETL, easy for BI users	Easier to update one attribute
Default choice?	**Yes** — storage is cheap, simplicity wins	Only for very large or shared dimensions

Vocabulary interviewers expect

Fact table types

- **Transaction** — one row per event.
- **Periodic snapshot** — one row per entity per period (daily balance).
- **Accumulating snapshot** — one row per process, updated as it progresses (order lifecycle).
- **Factless** — records that an event occurred (attendance).

Dimension concepts

- **Surrogate key** — meaningless integer PK, insulates from source-key changes.
- **Conformed** dimension shared across facts (same dim_date everywhere).
- **Role-playing** — dim_date as order date, ship date, due date.
- **Degenerate** dimension — an ID kept in the fact (order_number).
- **Junk** dimension — bundles low-cardinality flags.

REMEMBER

Measures are **additive** (revenue — sum across everything), **semi-additive** (account balance — sum across accounts, not across time) or **non-additive** (ratios, percentages — recompute from components).

1.11 Slowly Changing Dimensions (SCD)

★★★

IN ONE LINE

Techniques for tracking how descriptive attributes of a dimension change over time.

Dimension attributes change (a customer moves city, a product is re-categorised). **Slowly Changing Dimensions** define how history is handled. Example: customer *Asha* moves from Pune to Mumbai in July 2025.

TYPE 1 · OVERWRITE

id	name	city	
C1	Asha	Mumbai	← was Pune

No history kept. Simple; use for typo fixes.

TYPE 3 · PREVIOUS COLUMN

id	name	city	prev_city
C1	Asha	Mumbai	Pune

Only one prior value. Rarely used.

TYPE 2 · NEW ROW PER CHANGE (default for history)

surrogate_key	cust_id	name	city	valid_from	valid_to	is_current
101	C1	Asha	Pune	2023-01-01	2025-06-30	N
102	C1	Asha	Mumbai	2025-07-01	9999-12-31	Y

① **expire the old row (close valid_to, is_current = N)**
② **insert a new row with a NEW surrogate key (valid_to = 9999-12-31, is_current = Y)**

Fact rows keep pointing at the key that was current when they happened → "as-was" reporting.

Figure 1.11 — SCD Types 1, 2 and 3 for the same change

Type	Behaviour	History?	Use when
0	Never changes (retain original)	n/a	Birth date, original sign-up channel
1	Overwrite old value	× none	Corrections; history is irrelevant
2	Add a new row, version with dates / flag	✓ full	**Most common** — audit and point-in-time analytics
3	Add a column holding the previous value	◐ one step	Compare current vs prior only
4	Current table + separate history table	✓ full	Large, fast-changing dimensions
6	Hybrid 1 + 2 + 3	✓ full	Need current value on every historical row

Typical SCD2 load (Delta / SQL pattern)

SQL · SCD TYPE 2

```sql
-- 1) close rows whose tracked attributes changed
MERGE INTO dim_customer t
USING stg_customer s
  ON t.cust_id = s.cust_id AND t.is_current = true
WHEN MATCHED AND t.city <> s.city THEN
  UPDATE SET t.valid_to = current_date() - 1, t.is_current = false;

-- 2) insert new current versions (changed + brand-new customers)
INSERT INTO dim_customer
SELECT uuid(), s.cust_id, s.name, s.city, current_date(), DATE '9999-12-31', true
FROM stg_customer s
LEFT JOIN dim_customer t ON t.cust_id = s.cust_id AND t.is_current = true
WHERE t.cust_id IS NULL OR t.city <> s.city;
```

INTERVIEW ANGLE

Follow-up: *"How do you join a fact to an SCD2 dimension?"* — either by **surrogate key** captured at load time, or by natural key plus `fact.event_date BETWEEN valid_from AND valid_to`. dbt offers **snapshots**, and Lakeflow declarative pipelines offer `APPLY CHANGES INTO ... STORED AS SCD TYPE 2`, to automate this.

02

Part 2 · Storage, Platforms & Architecture

Where analytical data lives: warehouses, lakes, lakehouses, open formats, engines, and the organisational patterns around them.

- Data Warehouse · Lake · Lakehouse
- Parquet
- Open Table Formats · Delta · Transaction Log
- Apache Spark · Trino
- Lakebase
- Data Mart · Mesh · Fabric
- Medallion (Bronze, Silver, Gold)

2.1 Data Warehouse

★★★

IN ONE LINE

A central store of integrated, historical, structured data modelled for fast analytical queries.

A data warehouse is a central, **modelled** repository of cleaned, historical business data optimised for analytics. Data is transformed *before* it is loaded (**schema-on-write**), so users get consistent, fast SQL on trustworthy tables.

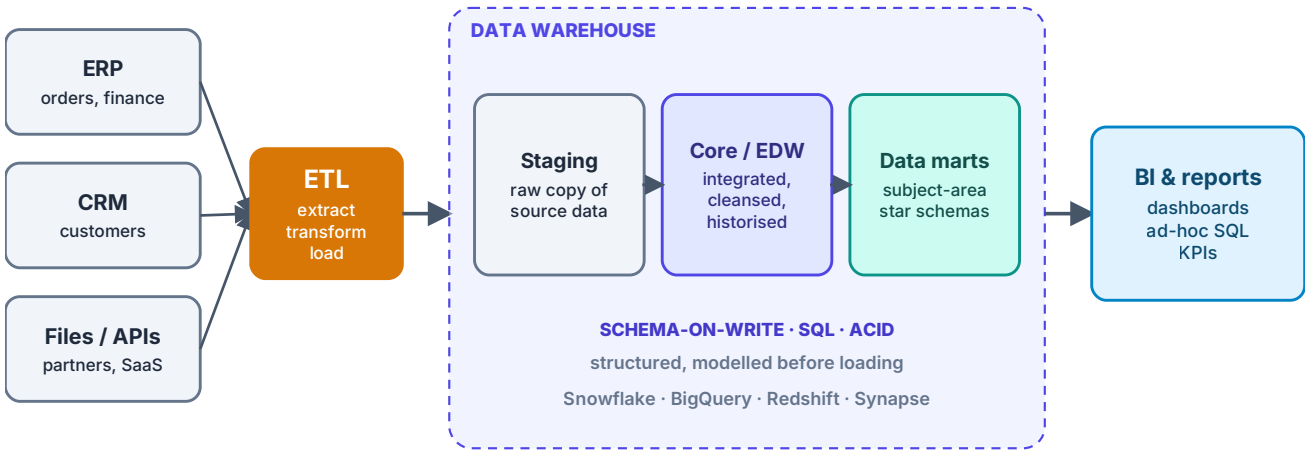


Figure 2.1 — Classic data warehouse architecture

Inmon's four traits

- **Subject-oriented** — organised by business topic (sales), not by app.
- **Integrated** — one consistent version of codes, units, keys.
- **Time-variant** — keeps history for trend analysis.
- **Non-volatile** — data is added, not overwritten by day-to-day ops.

Two schools

- **Inmon (top-down)** — build a normalised enterprise warehouse first, then derive marts from it.
- **Kimball (bottom-up)** — build dimensional marts with conformed dimensions (the "bus"); the union is the warehouse.

COMMON TRAP

Classic warehouses couple storage and compute, store mostly structured data and can be expensive or proprietary. That limitation is exactly why data lakes appeared — and why lakehouses merged the two ideas.

2.2 Data Lake

★★★

IN ONE LINE

Cheap, scalable storage that holds raw data in its native format, with structure applied when read.

A data lake stores **raw data of any type** in cheap object storage with no upfront schema (**schema-on-read**). It is flexible and scales to petabytes, and is the natural home for logs, clickstreams, images and ML training data.

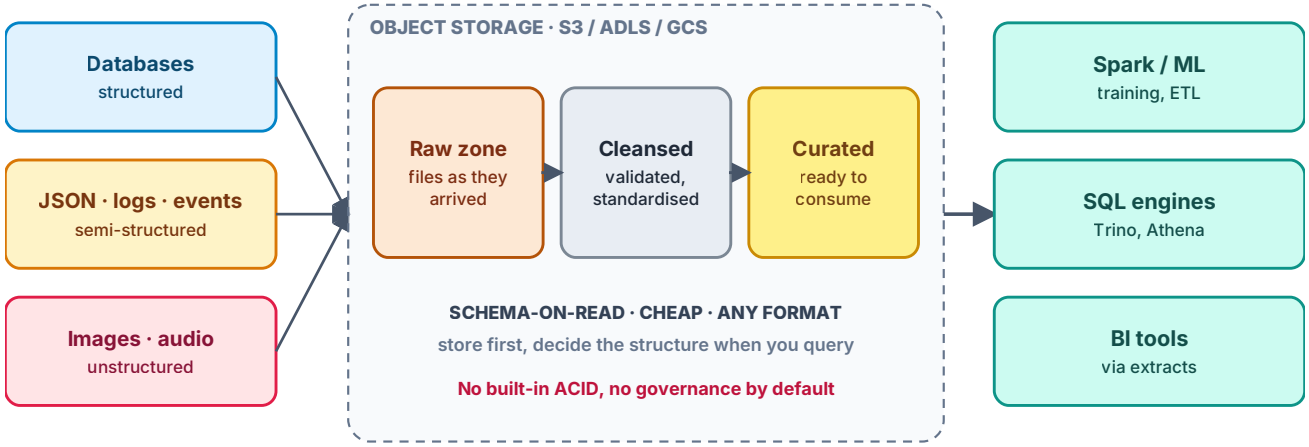


Figure 2.2 — Data lake: land everything, structure later

Strengths

- Very low cost per TB.
- Stores structured, semi- and unstructured data.
- Decoupled storage and compute.
- Ideal for data science and ML.

Weaknesses

- No ACID: failed writes leave partial files.
- No updates / deletes without rewriting files.
- Weak governance, quality and discoverability.
- Small-file and performance problems.

COMMON TRAP

Data swamp — a lake with no catalog, ownership, quality checks or naming standards becomes a dumping ground nobody trusts. The fix is metadata, governance and a layered design (see Medallion).

2.3 Data Lakehouse

★★★

IN ONE LINE

One open architecture combining the low-cost flexibility of a lake with the reliability and speed of a warehouse.

A lakehouse puts **warehouse-grade reliability and SQL performance on top of lake storage** using an open table format. You keep one copy of the data in open Parquet files and get transactions, governance and BI speed — instead of copying data between a lake and a warehouse.

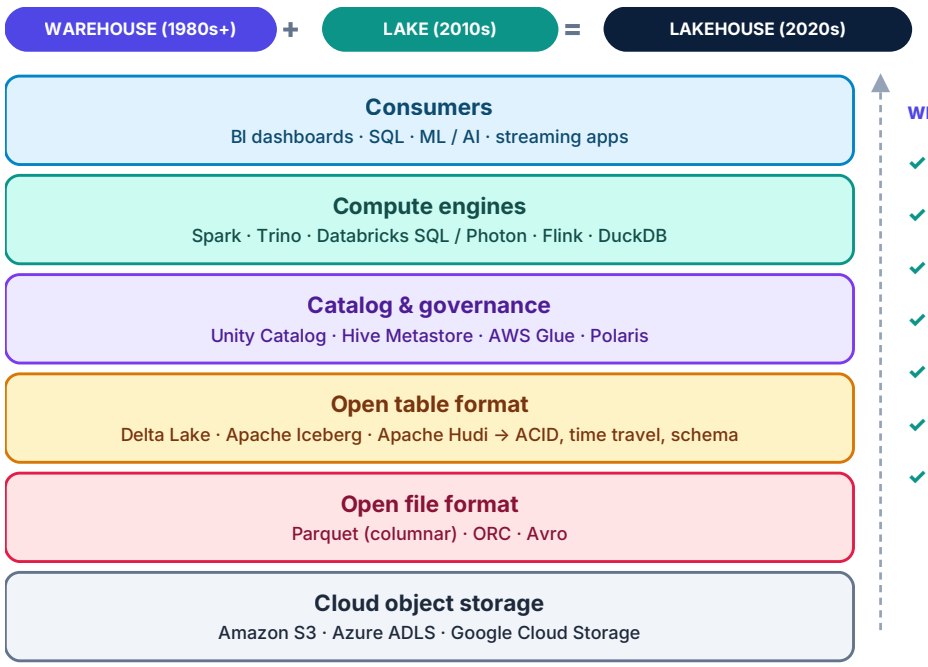


Figure 2.3 — The layered lakehouse stack

	Data warehouse	Data lake	Data lakehouse
Data types	Structured	All types	All types
Schema	On write (strict)	On read (flexible)	Enforced *and* evolvable
ACID	Yes	No	Yes (table format)
Cost / scale	Higher, proprietary storage	Lowest, object storage	Low, object storage
BI / SQL	Excellent	Weak	Excellent
ML / data science	Limited	Excellent	Excellent
Governance	Strong	Weak	Strong (catalog)

INTERVIEW ANGLE

A crisp answer to **"why lakehouse?"**: *it removes the two-tier lake-plus-warehouse architecture, which duplicated data, created stale copies and needed fragile ETL between them.* The enabling technology is the open table format with a transaction log.

2.4 Parquet (Columnar File Format)

★★★

IN ONE LINE

A compressed, columnar, self-describing file format designed for fast analytical reads.

Parquet is the de-facto **columnar file format** of the big-data world. A file is split into **row groups**; inside each, every column is stored contiguously as a **column chunk** made of compressed pages. A **footer** holds the schema and per-chunk statistics, so engines can skip data they do not need.

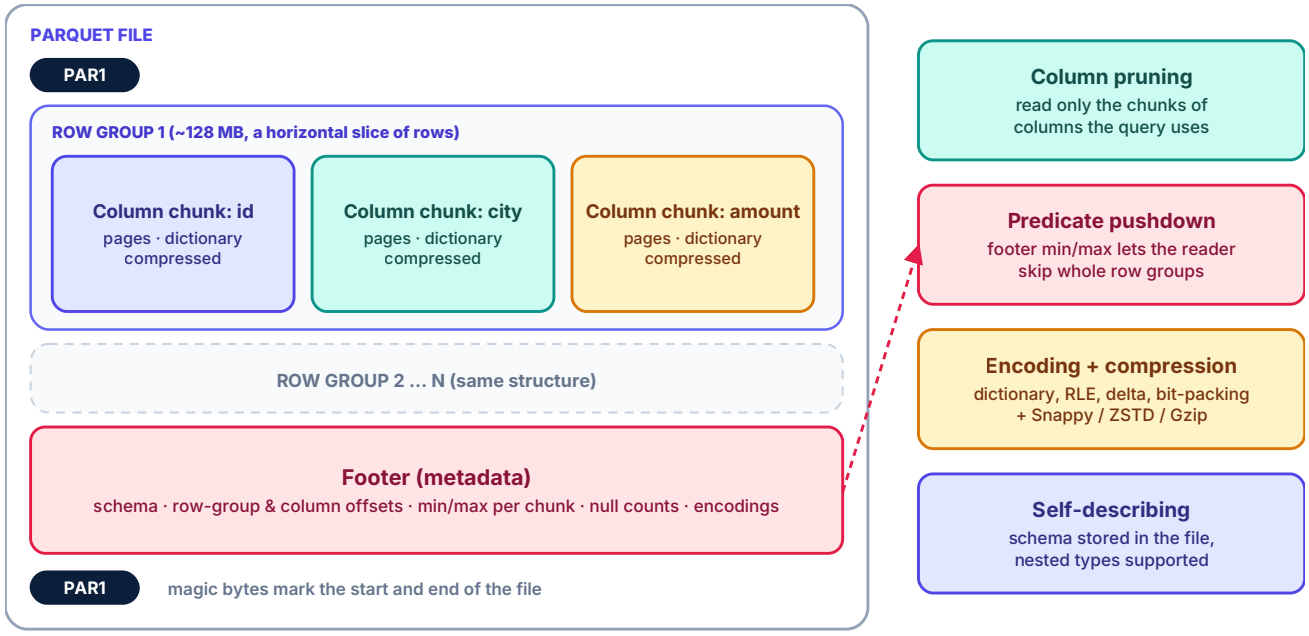


Figure 2.4 — Anatomy of a Parquet file

Format	Layout	Schema	Best for
CSV	Row, text	None (inferred)	Simple exchange; human readable; slow and ambiguous
JSON	Row, text	Implicit, nested	APIs, logs, semi-structured landing data
Avro	Row, binary	Embedded, evolvable	**Streaming / Kafka** messages, write-heavy ingestion
ORC	Columnar, binary	Embedded	Hive ecosystem, strong compression
Parquet	Columnar, binary	Embedded, nested	**Analytics**, lakehouse default

INTERVIEW ANGLE

Common follow-up: *"Parquet vs Delta?"* Parquet is just a **file format**; Delta is a **table format** = Parquet files *plus* a transaction log. Parquet files alone cannot give ACID, time travel or safe concurrent writes. Also be ready to explain that **many tiny Parquet files hurt performance** (metadata overhead, too many tasks).

2.5 Open Table Formats, Delta Lake & the Transaction Log

★★★

IN ONE LINE

A metadata layer (usually a log) that gives ACID tables, time travel and safe concurrency on top of plain Parquet files.

An **open table format** is a metadata layer that turns a pile of immutable files into a **table**: it tracks which files belong to the table at each point in time, what the schema is, and gives transactions on object storage. The three major formats are **Delta Lake**, **Apache Iceberg** and **Apache Hudi**.

Delta Lake & the transaction log

A Delta table is a directory of Parquet files plus a `_delta_log` folder. Every change (INSERT, MERGE, OPTIMIZE ...) appends a numbered JSON **commit** listing `add` and `remove` file actions. The current state is simply *the replay of the log*. Every ~10 commits a **checkpoint** Parquet file summarises the state so readers need not replay thousands of JSON files.

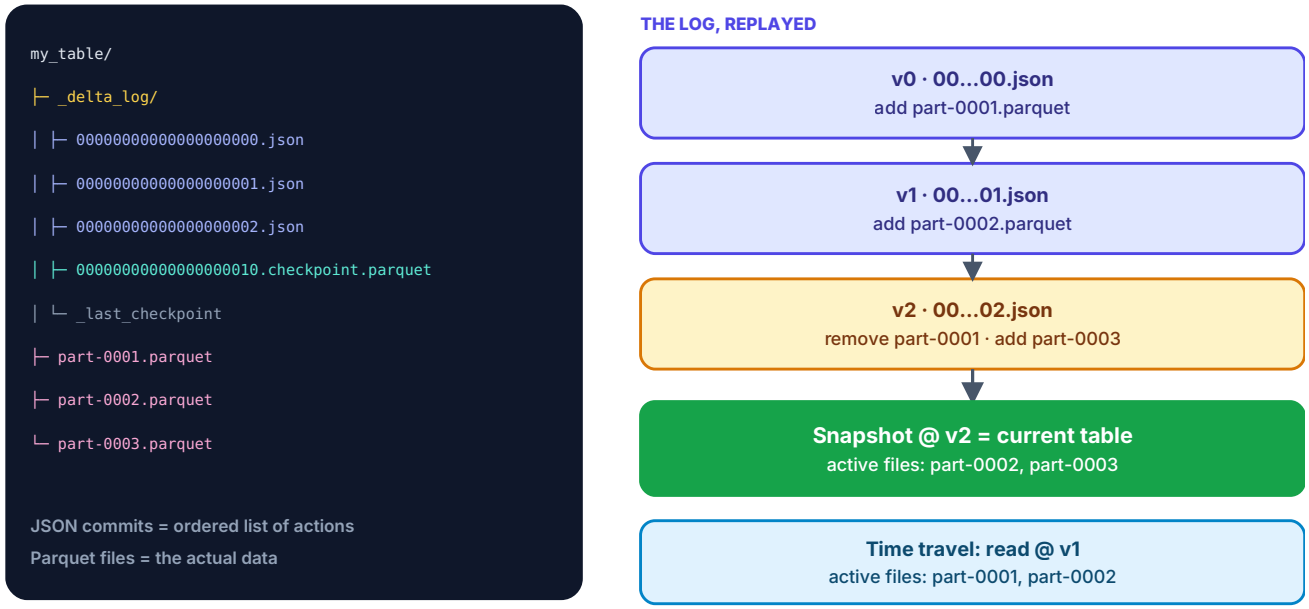
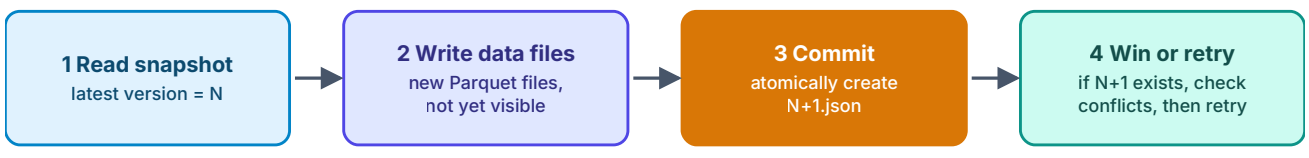


Figure 2.5 — A Delta table on disk and how the log defines each version



Optimistic concurrency: writers do not lock; the first to create the next log file wins.

Readers always see a consistent snapshot, so reads never block writes.

Figure 2.6 — How a Delta write commits atomically (optimistic concurrency)

What the log enables

- **ACID** on object storage — readers only trust files named in a committed version.
- **Time travel** — query or restore any earlier version.
- **Schema enforcement & evolution.**
- **Efficient upsert / delete** with `MERGE`.
- **Change Data Feed** — row-level changes between versions.

Housekeeping commands

- `OPTIMIZE` — compact small files.
- `ZORDER BY` / **liquid clustering** — co-locate related data for skipping.
- `VACUUM` — delete files no longer referenced (breaks time travel past the retention).
- `DESCRIBE HISTORY` — audit the log.

SQL · DELTA LAKE

```sql
-- time travel
SELECT * FROM sales VERSION AS OF 12;
SELECT * FROM sales TIMESTAMP AS OF '2026-10-01';
RESTORE TABLE sales TO VERSION AS OF 12;

-- maintenance
OPTIMIZE sales ZORDER BY (customer_id);
VACUUM sales RETAIN 168 HOURS;
DESCRIBE HISTORY sales;
```

	Delta Lake	Apache Iceberg	Apache Hudi
Metadata	JSON log + Parquet checkpoints	metadata.json → manifest lists → manifests	Timeline of commits + metadata table
Origin	Databricks	Netflix	Uber
Known for	Tight Spark / Databricks integration, MERGE, CDF	Engine-neutral, hidden partitioning, partition evolution	Upserts, incremental pulls, merge-on-read
Interop	UniForm can expose Delta tables to Iceberg readers	Wide engine and catalog (REST) support	Spark, Flink, Presto / Trino

COMMON TRAP

Never run `VACUUM` with a tiny retention (e.g. 0 hours) while long-running readers or streams are active — they may need files you just deleted. The default 7-day retention exists for a reason.

2.6 Apache Spark

★★★

IN ONE LINE

A distributed engine that processes large datasets in parallel across a cluster, with lazy, optimised execution.

Apache Spark is a **distributed, in-memory data-processing engine** for batch, streaming, SQL and ML. You describe *what* you want with DataFrames / SQL; Spark builds an optimised plan and runs it in parallel across a cluster.

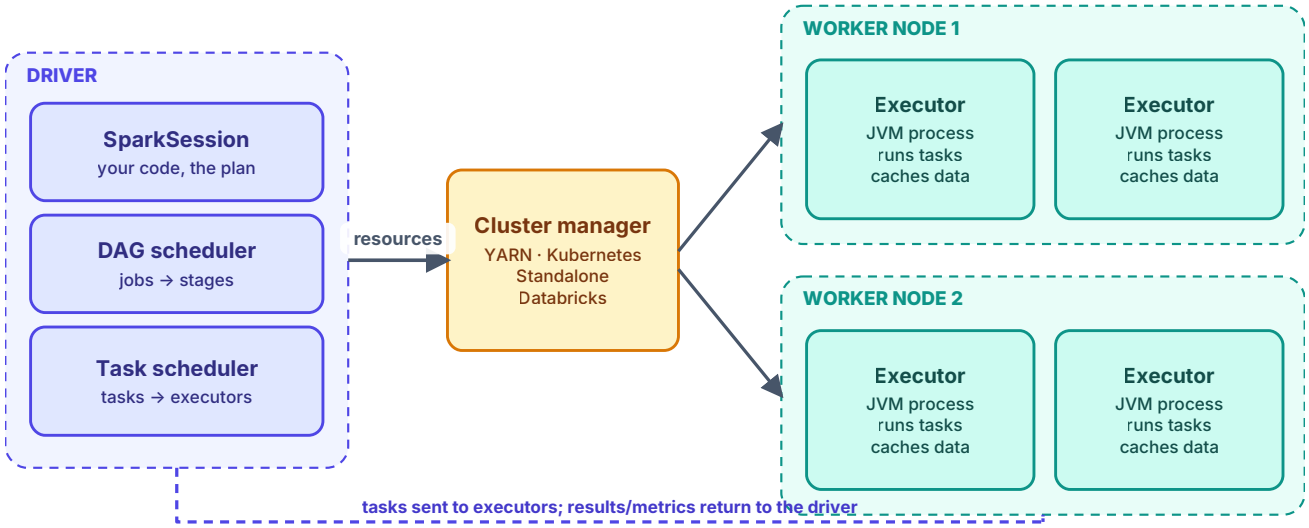
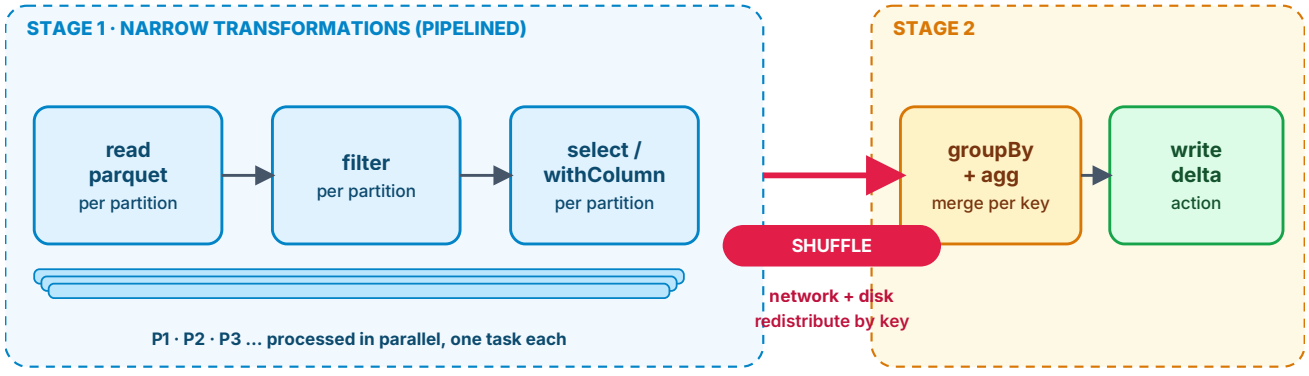


Figure 2.7 — Spark architecture: driver, cluster manager, executors



An ACTION (write / count / collect) triggers the whole DAG; a shuffle is the boundary between stages.

Figure 2.8 — From code to stages and tasks

Concept	Meaning
Lazy evaluation	**Transformations** (select, filter, join, groupBy) only build a plan. **Actions** (count, collect, write, show) trigger execution.
Narrow vs wide	Narrow: each output partition depends on one input partition (filter, map) — no data movement. Wide: needs a **shuffle** (groupBy, join, distinct, orderBy).
Job → Stage → Task	An action creates a job; shuffles split it into stages; each stage runs one **task per partition**.
Catalyst & Tungsten	Catalyst optimises the logical plan (predicate pushdown, column pruning, join reordering); Tungsten generates efficient code and manages memory.
AQE	**Adaptive Query Execution** re-optimises at runtime: coalesces shuffle partitions, switches join strategy, splits skewed partitions.
Partitions	The unit of parallelism. `repartition(n)` shuffles to n; `coalesce(n)` only reduces, without a full shuffle.

PYSPARK

```pyspark
from pyspark.sql import functions as F

df = (spark.read.format('delta').load('/lake/bronze/orders')   # lazy
      .filter(F.col('status') == 'PAID')                       # narrow
      .groupBy('customer_id')                                  # wide -> shuffle
      .agg(F.sum('amount').alias('total')))

df.write.format('delta').mode('overwrite').save('/lake/gold/spend')   # ACTION
```

INTERVIEW ANGLE

The best performance answers are about **shuffles**: broadcast the small side of a join, filter early, avoid Python UDFs (use built-in functions), pick sensible partition counts (`spark.sql.shuffle.partitions` defaults to 200), and fix skew. We expand on this in Part 4.

2.7 Trino

★★☆

IN ONE LINE

A distributed MPP SQL engine that queries data in place across many sources with low latency.

Trino (formerly PrestoSQL) is a **distributed SQL query engine** for fast, interactive analytics. It stores no data itself; **connectors** let it query data where it already lives — lake files, relational databases, Kafka — and even **join across them** in a single query (*federation*).

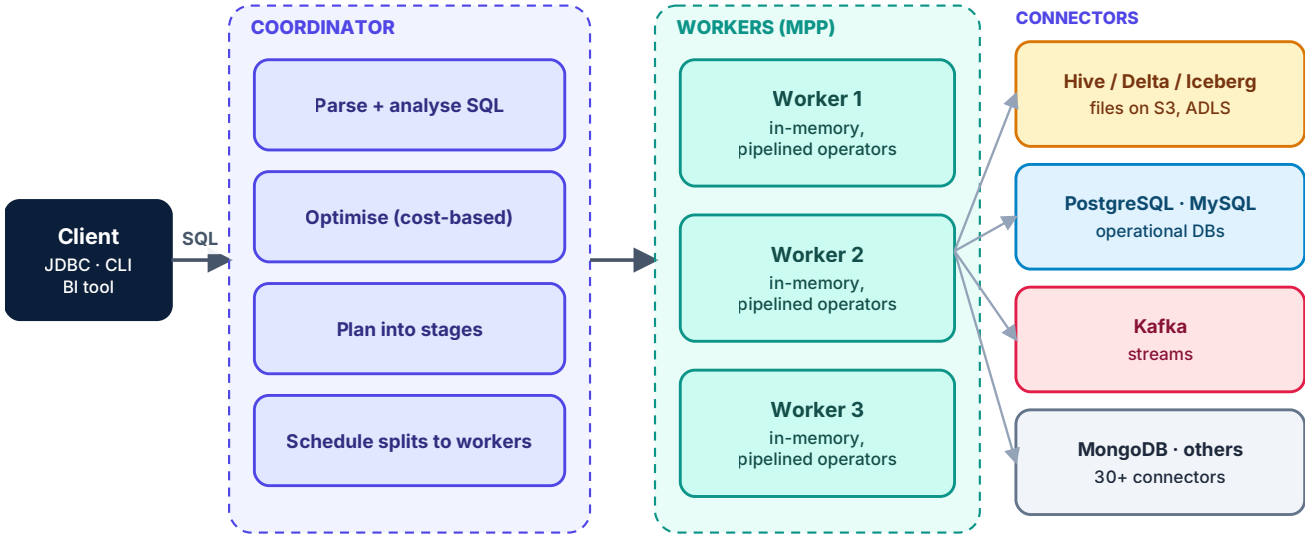


Figure 2.9 — Trino: coordinator, workers and connectors

TRINO · FEDERATED QUERY

```
-- one query, two systems
SELECT c.segment, SUM(o.amount) AS revenue
FROM hive.sales.orders o              -- Parquet on S3
JOIN postgresql.crm.customers c       -- live Postgres table
  ON o.customer_id = c.id
WHERE o.order_date >= DATE '2026-01-01'
GROUP BY c.segment;
```

	Apache Spark	Trino
Primary role	Large-scale ETL / ELT, streaming, ML	Fast interactive SQL, ad-hoc analytics, federation
Execution	Stage-based with shuffles to disk; resilient (task retries)	Pipelined MPP in memory; very low latency
Fault tolerance	Built in (lineage / task retry)	Queries traditionally fail on node loss; optional fault-tolerant mode exists
Languages	SQL, Python, Scala, Java, R	SQL only
Writes	Rich (batch + streaming writers)	Supported via connectors (e.g. Iceberg / Delta), mostly for SQL transforms
Pick it when	Heavy transformations, pipelines, ML	Analysts need seconds-latency SQL across many sources

INTERVIEW ANGLE

A neat one-liner: **"Spark is the workhorse that builds your tables; Trino is the query layer that explores them."** Both read the same Delta / Iceberg tables in a lakehouse.

2.8 Lakebase ★★☆

IN ONE LINE

Databricks' serverless Postgres OLTP database, integrated with the lakehouse so apps and analytics share governed data.

Lakebase is Databricks' **fully managed, Postgres-compatible OLTP database** built into the lakehouse platform. It is based on the serverless Postgres technology from **Neon** (acquired by Databricks in 2025) and separates compute from storage, so it can autoscale, scale to zero and offer database-style *branching*.

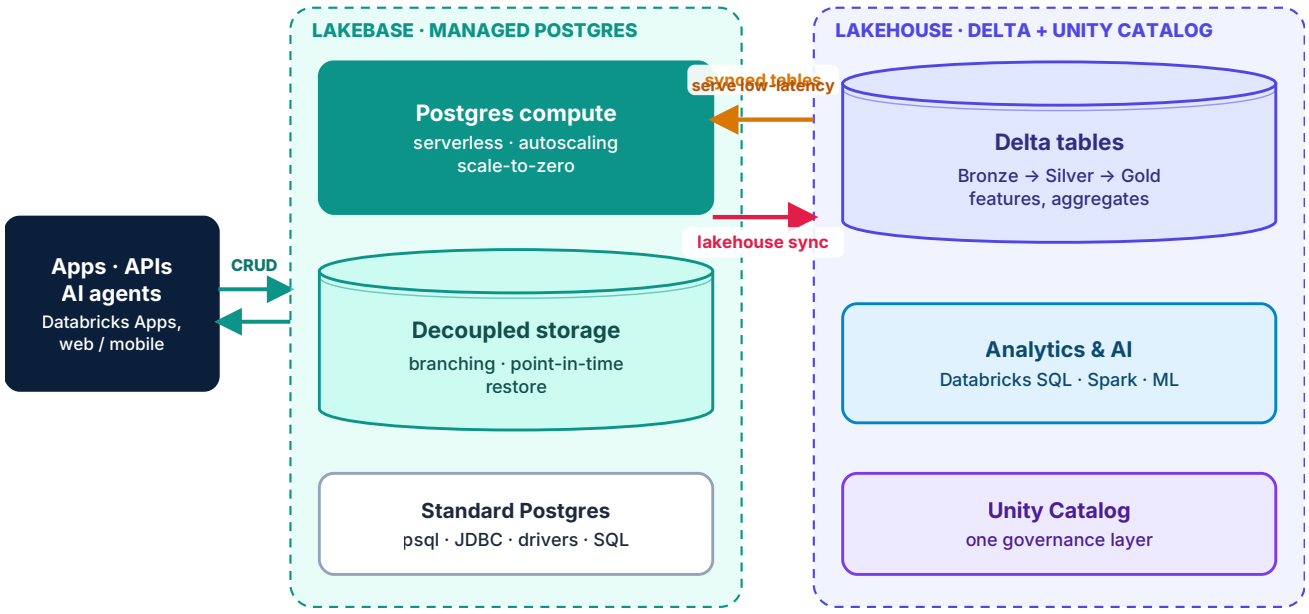


Figure 2.10 — Lakebase sits on the operational side and is wired to the lakehouse

Why it exists

Historically the OLTP database (RDS, Aurora, Cloud SQL) lived *outside* the analytics platform, so teams built fragile ETL to move data in and reverse-ETL to push insights back to apps. Lakebase puts transactions and analytics on one governed platform so both directions become managed sync rather than custom pipelines.

Typical uses

- Operational apps and APIs that need row-level transactions (Databricks Apps).
- Serving lakehouse features / model outputs with low latency (online feature store).
- State for AI agents and RAG applications.
- Replacing a separate managed Postgres next to the lakehouse.

Key ideas to name-drop

- **Synced tables** — a Unity Catalog / Delta table mirrored into Postgres for fast lookups.
- **Lakehouse Sync** — operational data flows back into Delta for analytics.
- **Standard Postgres** — existing drivers and tools work.
- Governed through **Unity Catalog**.

COMMON TRAP

Lakebase is **not** a replacement for your warehouse or lakehouse tables — it handles OLTP workloads next to them. Product names, regions and GA status are still evolving, so check the current Databricks documentation before quoting specifics in an interview.

INTERVIEW ANGLE

Frame it as the industry trend **"converging OLTP and OLAP"** (like HTAP), but done by *integration* (Postgres + lakehouse + sync) rather than a single hybrid engine.

2.9 Data Mart

★★☆

IN ONE LINE

A subject-specific slice of analytical data built for one team or business function.

A data mart is a **focused subset of analytical data for one business function or team** — typically a small star schema. It is smaller, faster to query, easier to secure, and speaks the business unit's own language.

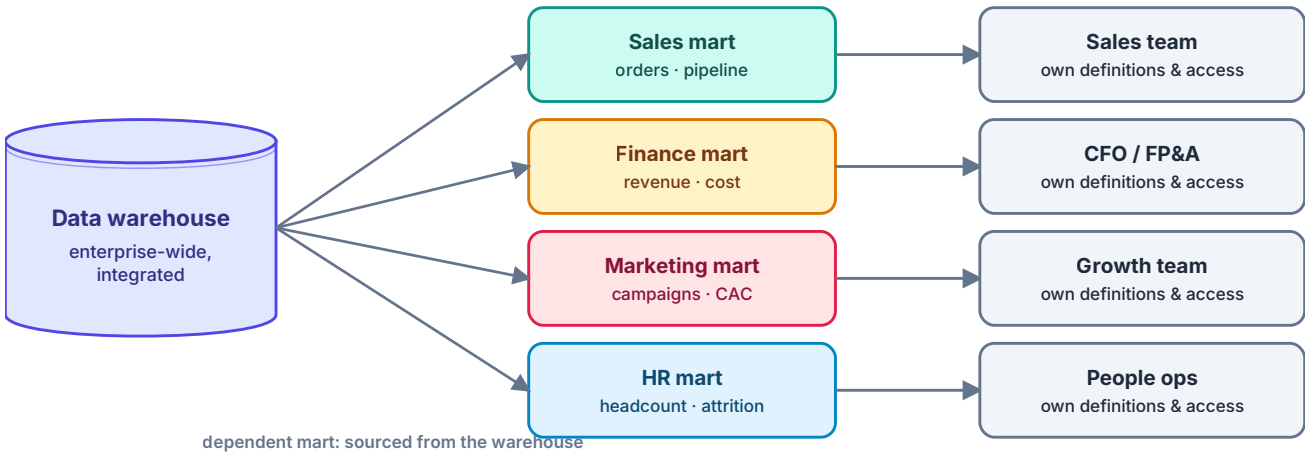


Figure 2.11 — Departmental data marts fed from the central warehouse

	Data warehouse	Data mart
Scope	Enterprise-wide, many subject areas	One department / subject area
Size	Large (TBs–PBs)	Small (GBs–low TBs)
Audience	Analysts, data scientists, org-wide	Business users in one function
Build time	Months	Weeks
Modelling	Normalised core and/or conformed dimensions	Dimensional (star schema)

Dependent mart

Built **from** the warehouse. Consistent numbers, single source of truth. (Inmon style.)

Independent mart

Built straight from sources. Quick, but risks "data silos" and conflicting KPIs unless dimensions are conformed. (Kimball bus keeps this safe.)

REMEMBER
In a lakehouse the **Gold layer is effectively the set of data marts**: curated, business-aligned tables per domain.

2.10 Data Mesh

★★★

IN ONE LINE
A decentralised approach where business domains own and publish data as products on a shared self-serve platform.

Data mesh (Zhamak Dehghani) is a **socio-technical approach** that fixes the bottleneck of a single central data team. Instead of funnelling everything into one pipeline team, data ownership is **decentralised to business domains**, who publish reliable data as products on a shared platform.

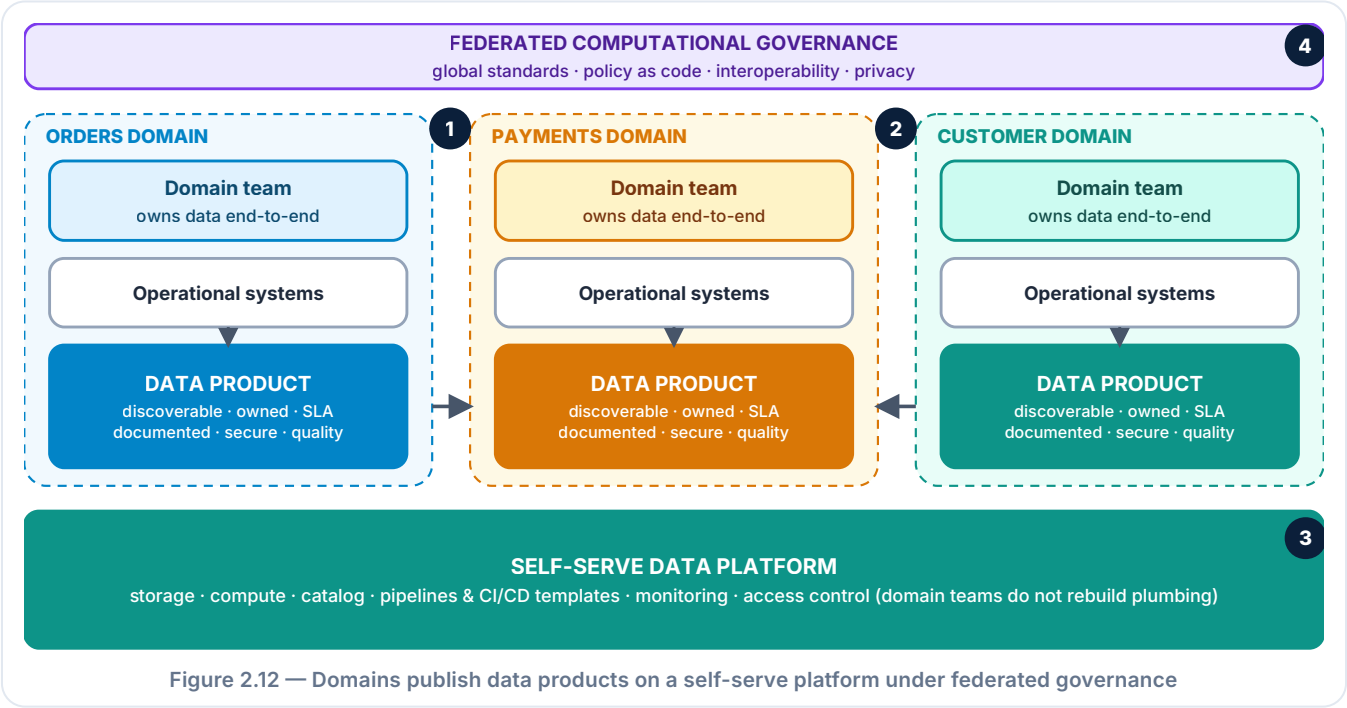


Figure 2.12 — Domains publish data products on a self-serve platform under federated governance

#	Principle	What it means in practice
1	Domain-oriented ownership	The team closest to the data (Orders, Payments…) owns, models and serves it.
2	Data as a product	Datasets have owners, SLAs, documentation, versioning, quality metrics and a consumer-first mindset.
3	Self-serve data platform	A platform team provides paved-road tooling so domains can publish without being infrastructure experts.
4	Federated computational governance	Global rules (security, naming, interoperability) are agreed centrally but **enforced by code** in the platform.

COMMON TRAP
Mesh is an *organisational* pattern, not a product you buy. It adds real overhead and suits large organisations with many autonomous domains; small teams are usually better served by a well-run central lakehouse.

2.11 Data Fabric

★★☆

IN ONE LINE

A metadata-driven, automated integration layer that unifies access to data across many systems.

A data fabric is a **technology architecture** that weaves distributed data sources together with an intelligent, **metadata-driven integration layer**. It continuously collects "active metadata" (lineage, usage, quality) and uses automation / AI to discover, connect, govern and serve data wherever it lives — on-prem, multi-cloud, lake or SaaS.

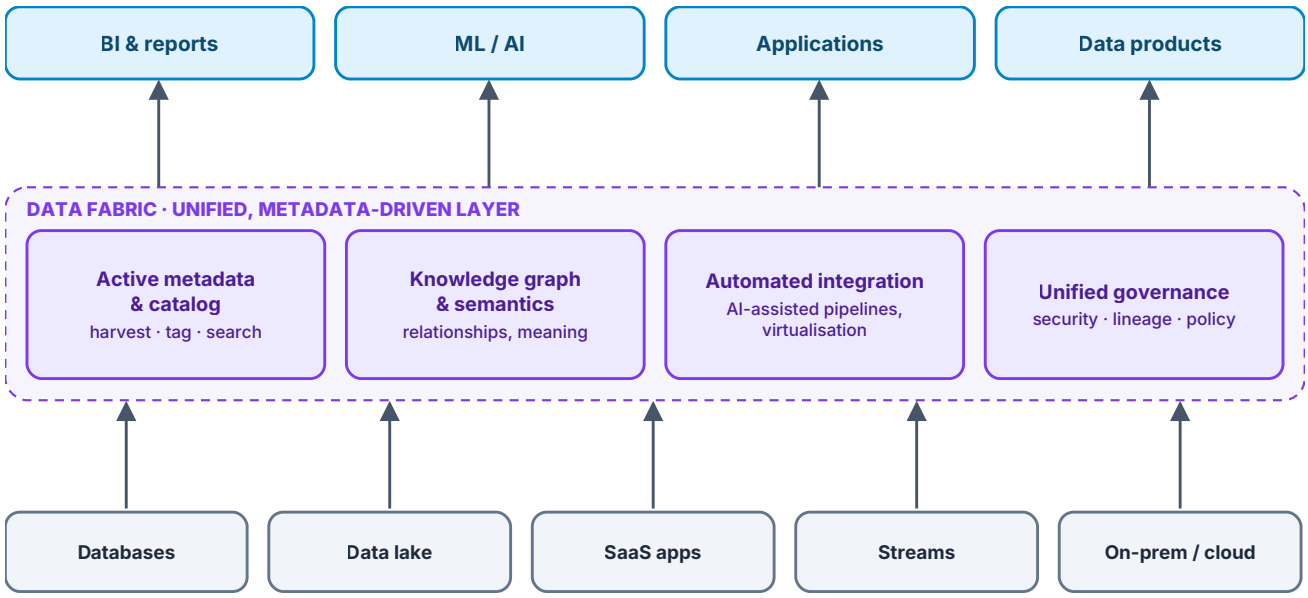


Figure 2.13 — Data fabric: a metadata and automation layer over heterogeneous sources

	Data mesh	Data fabric
Nature	Organisational / cultural	Technological / architectural
Centre of gravity	People and domain ownership	Metadata and automation
Data location	Stays with domains	Stays where it is; virtualised / integrated
Governance	Federated, domain-enforced	Centralised policies applied automatically
Can combine?	Yes — a fabric can be the platform that enables a mesh	

INTERVIEW ANGLE

The summary line: **mesh = "who owns the data", fabric = "how data is connected and automated".** They are complementary, not rivals.

2.12 Medallion Architecture (Bronze, Silver, Gold)

★★★

IN ONE LINE

A layered lakehouse design that progressively refines raw data into trusted, business-ready tables.

Medallion architecture organises a lakehouse into **three progressive quality layers**. Data is never "fixed in place": each hop reads one layer and writes the next, so you can always trace and rebuild downstream tables from the raw data.

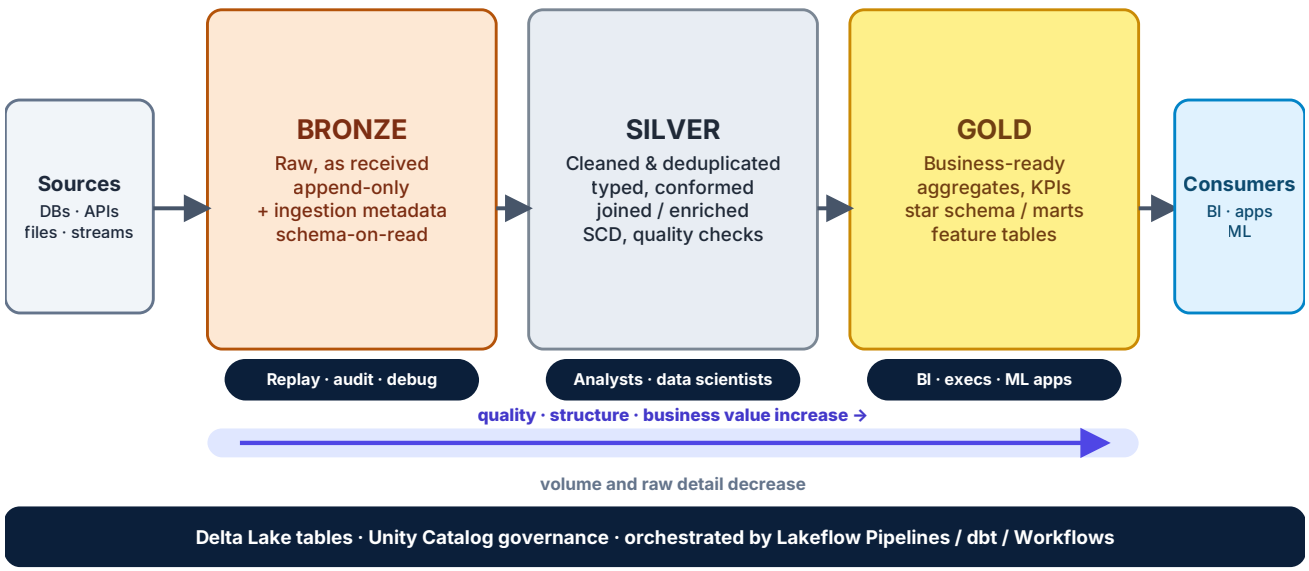


Figure 2.14 — Bronze → Silver → Gold

	Bronze	Silver	Gold
State of data	Raw copy, minimal change	Validated, deduped, conformed	Aggregated, modelled for use
Schema	As source (+ load time, source file)	Enforced, typed, standardised	Business model (star schema, wide tables)
Typical ops	Ingest (Auto Loader, CDC, Kafka)	Cleanse, join, apply CDC, SCD2, DQ rules	Aggregate, KPI logic, semantic definitions
Write pattern	Append	Merge / upsert	Overwrite or incremental merge
Consumers	Data engineers	Analysts, data scientists	BI, executives, ML applications

Why layers?

- Reprocess from Bronze after a bug — no need to re-pull the source.
- Clear data-quality contracts at each stage.
- Different access controls per layer.
- Incremental, restartable pipelines.

Variations

- Add a **quarantine** table for rows that fail checks.
- Some teams say raw / refined / curated, or add a **Platinum** / serving layer.
- Medallion is a *pattern*, not a product — it applies on any lakehouse.

INTERVIEW ANGLE

Interviewers often ask **“where would you deduplicate / apply business rules / mask PII?”**. Standard answers: dedupe and type-cast in **Silver**, business metrics in **Gold**, keep Bronze faithful to the source and mask PII on the way into Silver (or restrict Bronze access).

03

Part 3 · Pipelines, Processing & DataOps

How data moves and changes: loading patterns, correctness guarantees, streaming, CDC and the tooling that ships it.

- ETL vs ELT · Pipelines
- Incremental loading · Watermarks
- Back-date refresh · Idempotency
- Exactly-once · Upsert
- Batch vs Streaming
- Lambda vs Kappa
- CDC · Delta Live Tables
- dbt · CI/CD · DataOps

3.1 Data Pipelines: ETL vs ELT

★★★

IN ONE LINE

Automated flows that extract data, transform it, and load it — ETL transforms before loading, ELT after.

A **data pipeline** is an automated, repeatable flow that moves and shapes data from sources to destinations. The most common design question is **where the transformation happens**.

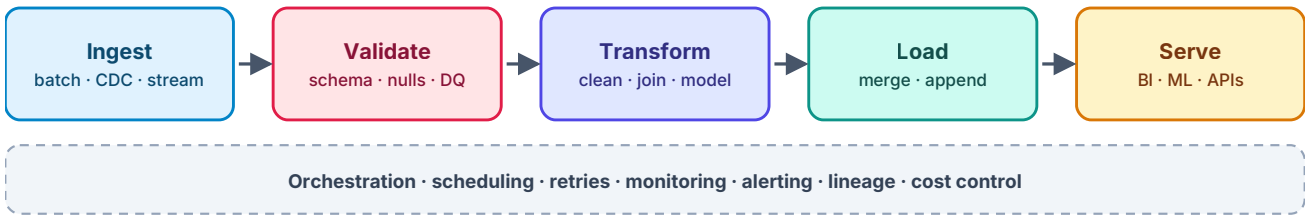


Figure 3.1a — Anatomy of a data pipeline

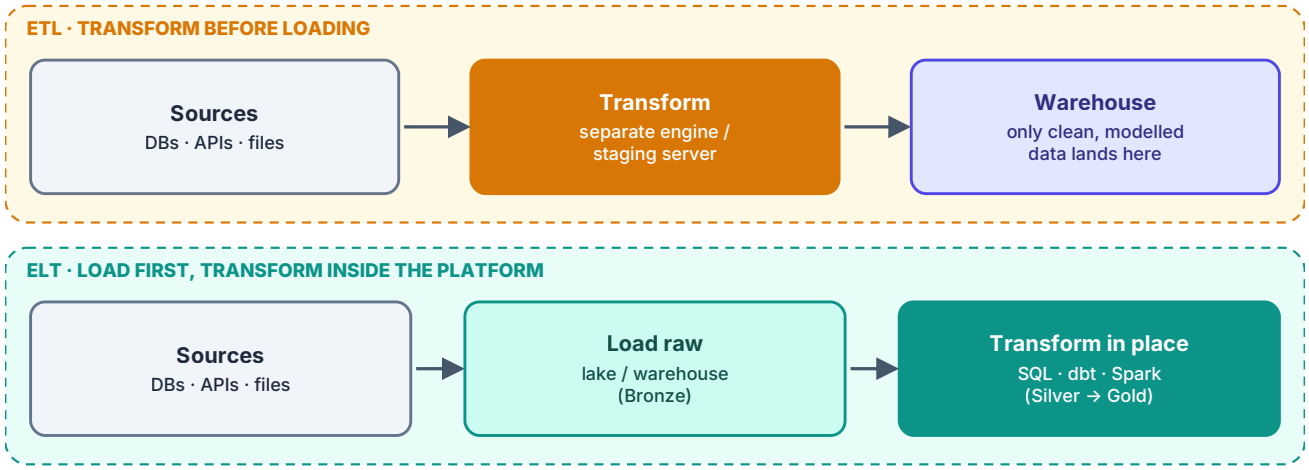


Figure 3.1b — ETL vs ELT

	ETL	ELT
Order	Extract → Transform → Load	Extract → Load → Transform
Where transform runs	Dedicated ETL tool / staging server	Inside the warehouse / lakehouse (SQL, dbt, Spark)
Raw data kept?	Usually not	**Yes** — reprocess any time
Fits	Strict compliance (mask before landing), legacy warehouses	Cloud warehouses / lakehouses with cheap, elastic compute
Flexibility	Rigid: schema decided up front	High: new models from the same raw data
Typical tools	Informatica, Talend, SSIS	Fivetran / Airbyte + dbt, Spark, Lakeflow

INTERVIEW ANGLE

Modern stacks are mostly **ELT** because storage and compute are cheap and decoupled. The honest nuance: *some* transformation (PII masking, light validation) often still happens before landing — "EtLT".

3.2 Incremental Data Loading

★★★

IN ONE LINE

Loading only the data that is new or changed since the previous run.

Instead of reloading everything every run (**full load**), an **incremental load** moves only new or changed data since the last run. It is faster, cheaper and the basis of almost every production pipeline.

SOURCE TABLE orders

id	amount	updated_at
1	500	10-17 09:00
2	300	10-17 18:30
3	750	10-18 08:10
4	120	10-18 11:45

green = changed after the watermark

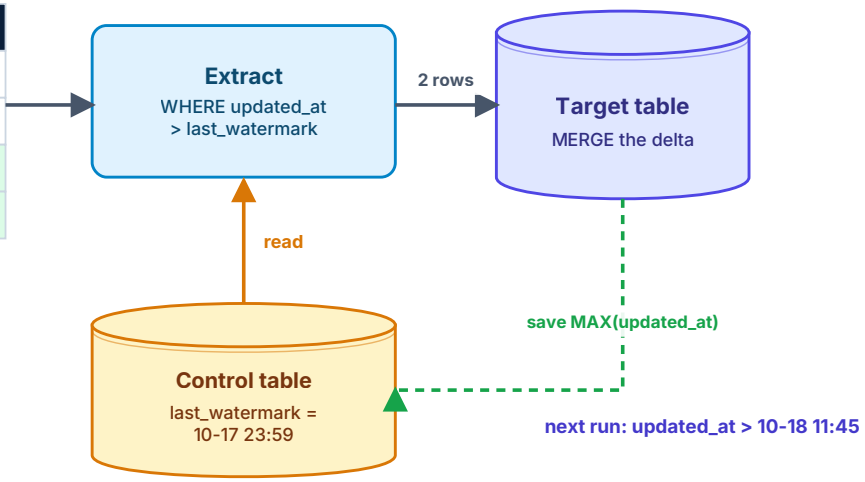


Figure 3.2 — High-water-mark incremental load

Pattern	How it works	Watch out for
Timestamp / watermark column	`WHERE updated_at > :last_ts`	Misses **hard deletes**; late commits; clock skew
Monotonic ID	`WHERE id > :last_id`	Only catches inserts, not updates
CDC (log-based)	Read inserts / updates / deletes from the DB log	More setup; ordering and initial snapshot
Partition / date based	Load yesterday's partition each day	Late-arriving data in old partitions
File based	Track which files were already ingested (Auto Loader, COPY INTO)	Files overwritten in place
Snapshot diff	Compare today's full snapshot with yesterday's	Expensive for large tables

SQL · INCREMENTAL PATTERN

```sql
-- 1) read the watermark   2) extract delta   3) merge   4) advance watermark
SELECT last_ts FROM etl_control WHERE table_name = 'orders';

MERGE INTO silver.orders t
USING (SELECT * FROM src.orders
       WHERE updated_at >= :last_ts - INTERVAL 1 HOUR) s   -- small overlap = safety margin
   ON t.id = s.id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;

UPDATE etl_control SET last_ts = (SELECT MAX(updated_at) FROM src.orders) WHERE table_name = 'orders';
```

COMMON TRAP

Use a **small overlap** (`>=` last watermark minus a margin) and make the load **idempotent** (MERGE). Otherwise rows committed late, or sharing the exact boundary timestamp, are silently lost.

3.3 Watermarks ★★★

IN ONE LINE

A marker of progress: the last loaded value in batch, or the point in event time the stream considers complete.

"Watermark" has **two different meanings** in data engineering — say which one you mean.

1 · Batch high-water mark

The stored value (a timestamp or ID) of the **last record successfully loaded**. The next incremental run extracts everything beyond it. It lives in a control table (see 3.2).

2 · Streaming event-time watermark

A moving threshold that says **"we assume no more events older than T will arrive"**. It tells the engine when a window can be finalised and when old state can be discarded.



#	event time	max seen	watermark (max − 5 min)	result
1	10:02	10:02	09:57	accepted
2	10:07	10:07	10:02	accepted
3	10:13	10:13	10:08	accepted · window 10:00–10:05 closes
4	10:03	10:13	10:08	LATE (10:03 < 10:08) → dropped

Figure 3.3 — Event-time watermark with 5 minutes of allowed lateness

Watermark = (maximum event time seen so far) − (allowed lateness). Once the watermark passes a window's end, the window is finalised and results are emitted; events arriving later than that are **dropped** (or routed to a side output). A longer lateness tolerates more disorder but increases latency and state size — the core trade-off.

SPARK STRUCTURED STREAMING

```
from pyspark.sql import functions as F

result = (events
  .withWatermark('event_time', '5 minutes')           # tolerate 5 min of lateness
  .groupBy(F.window('event_time', '5 minutes'), 'device_id')
  .agg(F.count('*').alias('n')))
```

REMEMBER

Event time = when the thing happened (in the payload). **Processing time** = when the system sees it. Networks and mobile clients make them differ; watermarks exist to bridge that gap.

3.4 Back-date Refresh (Lookback & Backfill)

★★★

IN ONE LINE

Re-processing past periods to include late data, corrections or new logic.

Data is rarely complete when first processed: events arrive late, upstream systems correct history, and bugs or logic changes require rebuilding the past. **Back-dated refresh** (a lookback window and/or a **backfill**) means re-processing historical periods and safely overwriting the old results.



Figure 3.4 — Normal run vs lookback window vs backfill (each box is a daily partition)

Scenario	Strategy
Late-arriving facts (mobile events, delayed files)	Rolling **lookback window**: each run re-processes the last N days, not just today.
Upstream correction / restatement	Re-extract the corrected range; **MERGE** or overwrite the affected partitions.
Logic change or bug fix	**Backfill** the historic range in parallel date-chunks after deploying the new code.
Late-arriving dimension	Insert an "unknown" placeholder member; update facts once the real dimension row lands.

PYSPARK · PARTITION-SCOPED OVERWRITE

```
# Delta: atomically replace only the affected days (idempotent!)
(df.write.format('delta')
   .mode('overwrite')
   .option('replaceWhere', "event_date >= '2026-10-16' AND event_date <= '2026-10-19'")
   .save('/lake/silver/events'))
```

INTERVIEW ANGLE

Orchestrators parameterise the run date (Airflow `{{ ds }}`, `catchup=True`, `airflow dags backfill`). The prerequisite for all of this is **idempotency** — running the same date twice must give the same result (next topic). Also mention **bitemporal** thinking: event date vs load date lets you answer "what did we know on day X?"

3.5 Idempotency ★★★

IN ONE LINE
Running a pipeline step multiple times yields the same result as running it once.

An operation is **idempotent** if running it once or many times produces the **same final result**. Pipelines *will* be retried (task failure, scheduler restart, backfill), so every step must be safe to re-run.

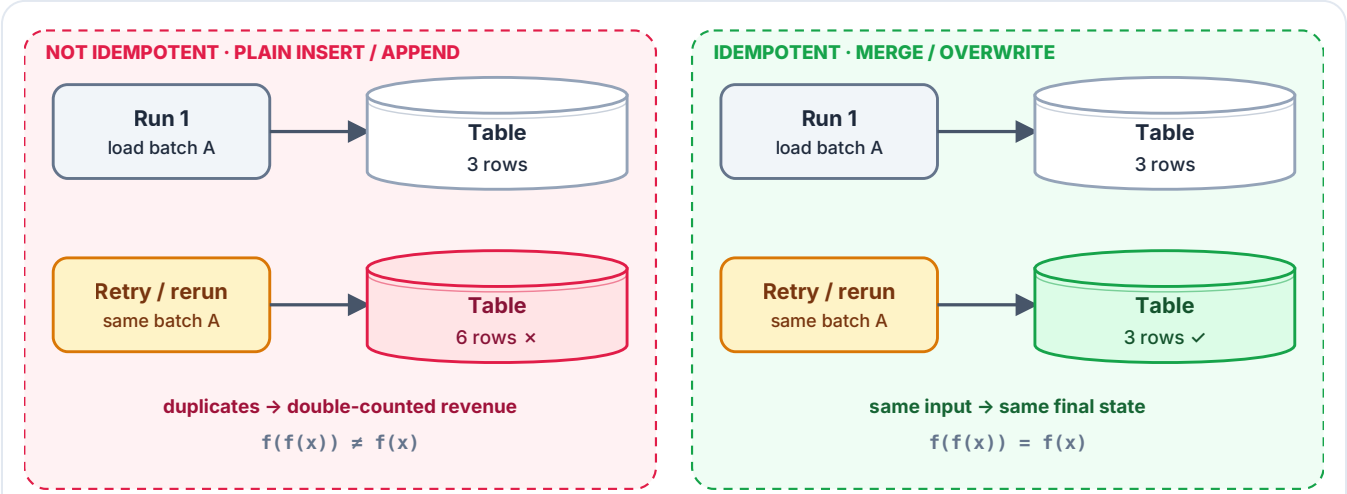


Figure 3.5 — Retrying a load: append duplicates, MERGE does not

Technique	How it makes a write idempotent
MERGE / upsert on a business key	Re-applying the same row updates it to the same values.
Overwrite the partition (`replaceWhere` , dynamic partition overwrite)	The run owns its slice: delete-then-write it atomically.
Deduplicate with a natural / deterministic key	`ROW_NUMBER() OVER (PARTITION BY key ORDER BY ts DESC) = 1`
Track processed units	Record file names / offsets / batch IDs already applied (Auto Loader, checkpoints, `txnAppId`).
Deterministic transforms	No `now()` or `random()` inside the logic; derive IDs by hashing keys.
Write-then-swap	Build in a temp location, then atomic rename / table swap.

COMMON TRAP
`INSERT INTO ... SELECT` and `mode("append")` are the classic non-idempotent writes. Also watch hidden state: `current_timestamp()` or auto-increment keys generated during the load change on every retry.

INTERVIEW ANGLE
Say this sentence: **“I design each task so it can be re-run for any date range without side effects — usually by partition overwrite or MERGE on a stable key.”**

3.6 Exactly-Once Processing

★★★

IN ONE LINE

Making sure each record's effect is applied once, despite retries and failures.

Messaging systems can promise three delivery semantics. **Exactly-once** is the hardest: in a distributed system with failures, a message can be sent once but processed twice (or vice versa). In practice it is achieved as **at-least-once delivery + idempotent (or transactional) processing** — the *effect* happens once even if the delivery repeats.

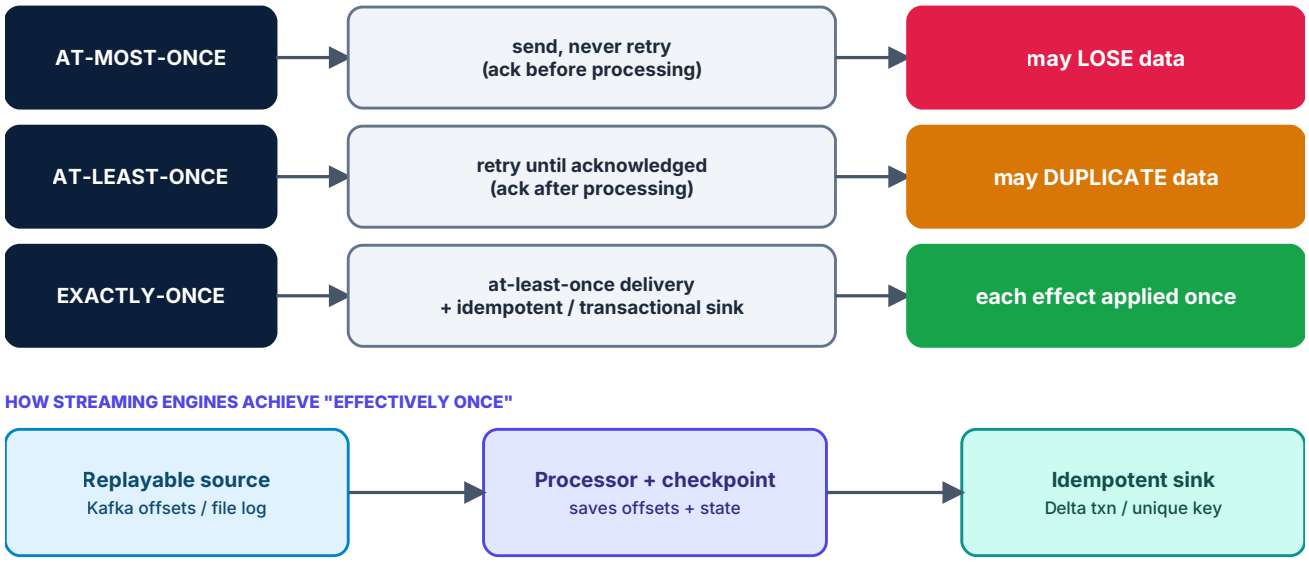


Figure 3.6 — Delivery semantics and the checkpoint-replay-dedupe recipe

Kafka

- **Idempotent producer** — broker de-duplicates retries per producer/partition.
- **Transactions** — atomic writes across partitions and offset commits (read-process-write).
- Consumers must read with `read_committed`.

Spark Structured Streaming + Delta

- **Checkpoint** stores source offsets and operator state.
- Replayable source + **idempotent Delta commits** (batch/txn IDs) = end-to-end exactly-once.
- `foreachBatch` sinks must be made idempotent by you (e.g. MERGE).

COMMON TRAP

"Exactly-once" only holds *within* the system that implements it. The moment you write to an external, non-idempotent sink (an email API, a payment call, a plain REST endpoint) you are back to at-least-once and must dedupe yourself with an idempotency key.

3.7 Upsert (MERGE)

★★★

IN ONE LINE

Update the row if the key exists, insert it if it does not — atomically.

Upsert = *update + insert*: if a row with the same key exists, update it; otherwise insert it. In SQL it is the `MERGE` statement. It is the workhorse behind incremental loads, CDC apply, SCD2 and idempotent pipelines.

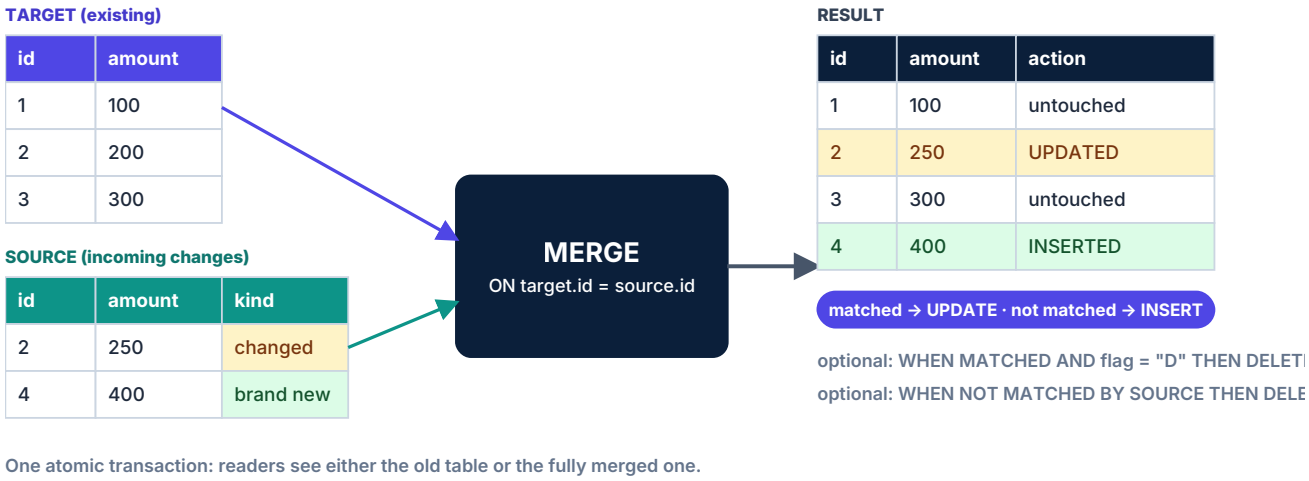


Figure 3.7 — MERGE: update matched rows, insert the rest

SQL · DELTA LAKE MERGE

```sql
MERGE INTO silver.customers AS t
USING updates AS s
   ON t.customer_id = s.customer_id
WHEN MATCHED AND s.op = 'D' THEN DELETE
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

Dialect names

- Delta / Snowflake / SQL Server / Oracle: `MERGE`
- PostgreSQL: `INSERT ... ON CONFLICT DO UPDATE`
- MySQL: `INSERT ... ON DUPLICATE KEY UPDATE`
- Mongo: `update` with `upsert: true`

Pitfalls

- **Duplicate keys in the source** → "multiple source rows matched" error or wrong results. De-duplicate first (latest per key).
- Unbounded `ON` clause scans the whole target — add partition / cluster filters.
- Merge rewrites files; many tiny merges cause small-file bloat.

INTERVIEW ANGLE

The standard CDC pattern is: **keep only the latest change per key from the batch, then MERGE**. Mention ordering by a sequence number / LSN, not by arrival time.

3.8 Batch vs Streaming Data Processing

★★★

IN ONE LINE

Batch handles bounded data on a schedule; streaming handles unbounded data continuously.

Batch processes a finite, bounded dataset on a schedule (hourly, nightly). **Streaming** processes an unbounded flow of events continuously, as they arrive. Many modern engines treat batch as a special case of streaming.

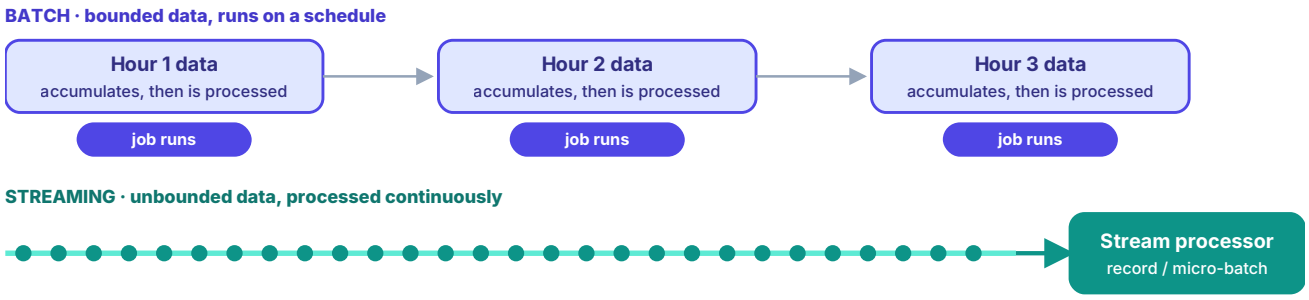


Figure 3.8a — Bounded vs unbounded processing

	Batch	Streaming
Latency	Minutes to hours	Milliseconds to seconds
Data	Bounded (a file, a day's partition)	Unbounded (events, clicks, sensor readings)
Complexity	Simpler to build, test, backfill	Harder: ordering, late data, state, exactly-once
Cost	Cheap — compute only while running	Always-on (or micro-batched) compute
Use when	Reports, ML training, nightly aggregates	Fraud detection, monitoring, real-time personalisation
Tools	Spark batch, dbt, SQL, Airflow	Kafka, Flink, Spark Structured Streaming, Kinesis

Windowing — turning an endless stream into aggregable chunks



Figure 3.8b — The three window types (dots are events)

Micro-batch vs continuous

Spark Structured Streaming defaults to **micro-batches** (latency ~100 ms–seconds); Flink is **record-at-a-time** (lower latency). A trigger like `availableNow` runs the streaming code as an incremental batch job — a cheap middle ground.

Correctness vocabulary

Event time vs **processing time**, **watermarks** for lateness, **stateful** operators, **checkpoints**, and **exactly-once** semantics.

INTERVIEW ANGLE

Do not say "streaming is always better". A mature answer: **pick the latency the business actually needs**; use batch (or `availableNow`) when hourly is fine, and streaming only where freshness has real value.

3.9 Lambda vs Kappa Architecture

★★★

IN ONE LINE

Two blueprints for serving both historical accuracy and real-time freshness.

Two classic patterns for combining **accuracy** (batch) with **freshness** (streaming).

Lambda architecture (Nathan Marz)

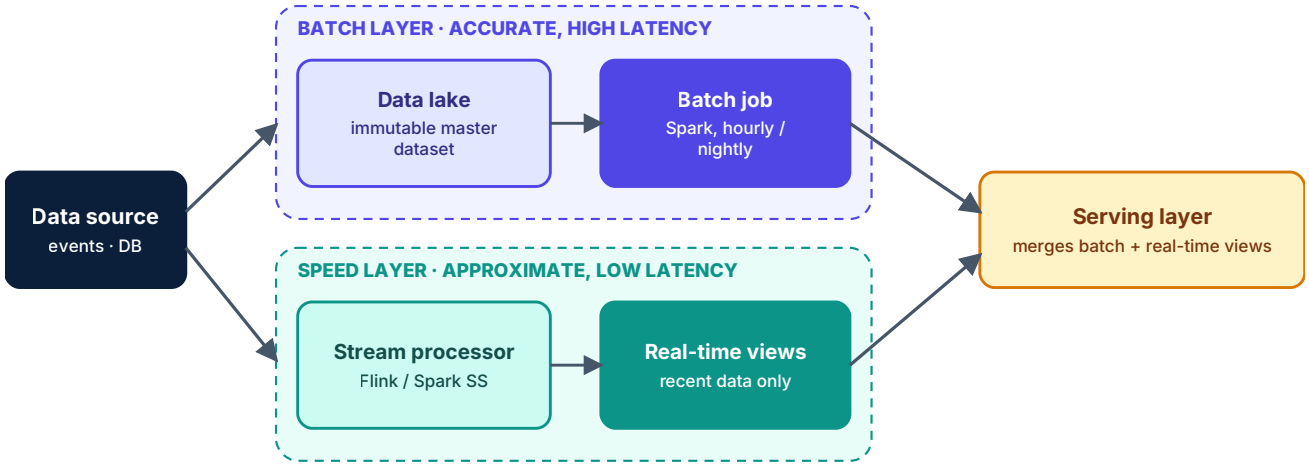


Figure 3.9a — Lambda: two parallel paths merged at serving time

Kappa architecture (Jay Kreps)

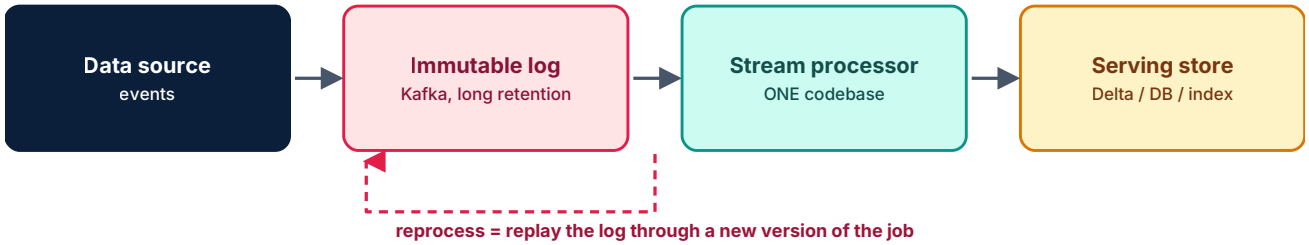


Figure 3.9b — Kappa: everything is a stream; the log is the source of truth

	Lambda	Kappa
Layers	Batch + speed + serving	Single streaming layer
Codebases	**Two** (must produce identical logic)	**One**
Reprocessing	Re-run batch job on the lake	Replay the log with new code
Pros	Accurate historical results; mature	Simple; one pipeline to maintain
Cons	Duplicate logic, drift, higher ops cost	Needs a long-retention replayable log; heavy streaming skills

INTERVIEW ANGLE

The modern answer: **lakehouse tables (Delta) unify both** — the same table is a streaming sink and a batch source with ACID, so you often get Kappa-like simplicity without keeping everything in Kafka. This is sometimes called the "Delta architecture" or "streaming-first lakehouse".

3.10 Change Data Capture (CDC)

★★★

IN ONE LINE

Capturing row-level inserts, updates and deletes from a source and propagating them downstream.

Change Data Capture identifies and delivers *row-level changes* (inserts, updates, **deletes**) from a source database to downstream systems, usually in near real time. Log-based CDC reads the same transaction log (WAL / binlog) the database uses for recovery — so it adds almost no load and captures every change in order.

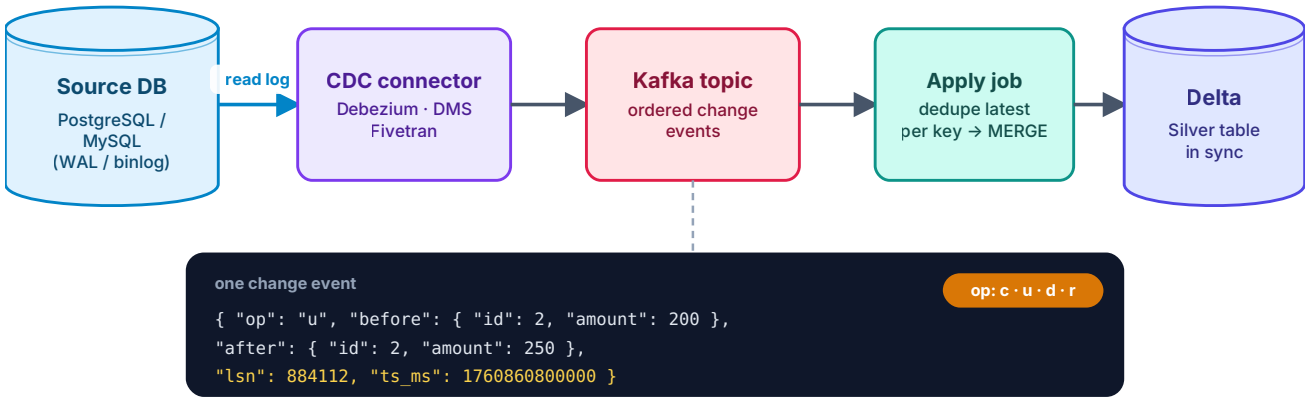


Figure 3.10 — Log-based CDC from an OLTP database into a Delta table

Method	How	Captures deletes?	Load on source
Log-based	Read WAL / binlog / redo log	✓ yes	Very low
Trigger-based	DB triggers write changes to an audit table	✓ yes	High (adds write overhead)
Query / timestamp	Poll `updated_at > last_ts`	× no (unless soft delete)	Medium (repeated queries)
Snapshot diff	Compare full extracts	✓ yes	High (full scans)

Implementation checklist

- **Initial snapshot** + then stream the log from a consistent position.
- Preserve **ordering** (LSN / sequence number) per key.
- Apply with **MERGE** after keeping the latest event per key.
- Handle **deletes** (hard delete or tombstone) and **schema changes**.
- Keep the raw change log in Bronze for replay.

Related tooling

- **Debezium** (open source, on Kafka Connect), AWS DMS, Fivetran, Airbyte.
- **Delta Change Data Feed** — row-level changes *between Delta table versions*, for downstream propagation.
- **Lakeflow** `APPLY CHANGES INTO` — declarative CDC apply with SCD1/SCD2 and out-of-order handling.

SQL · CHANGE DATA FEED

```sql
-- Delta Change Data Feed: what changed since version 5?
SELECT _change_type, * FROM table_changes('silver.customers', 5);
-- _change_type: insert | update_preimage | update_postimage | delete
```

INTERVIEW ANGLE

Tie it together: **WAL (1.4) → CDC (3.10) → MERGE (3.7) → SCD2 (1.11)**. This chain is a favourite “walk me through replicating a production DB to the lakehouse” question.

3.11 Delta Live Tables (Lakeflow Declarative Pipelines)

★★★

IN ONE LINE

A declarative framework: define tables and quality rules, and the engine manages dependencies, incremental runs and retries.

Delta Live Tables (DLT) is Databricks' **declarative** framework for building reliable batch and streaming pipelines: you declare the *tables you want* and the queries that define them, not the orchestration steps. Databricks has since renamed it **Lakeflow Spark Declarative Pipelines** (the engine was also contributed to Apache Spark as *Spark Declarative Pipelines*). Existing DLT code still runs unchanged.

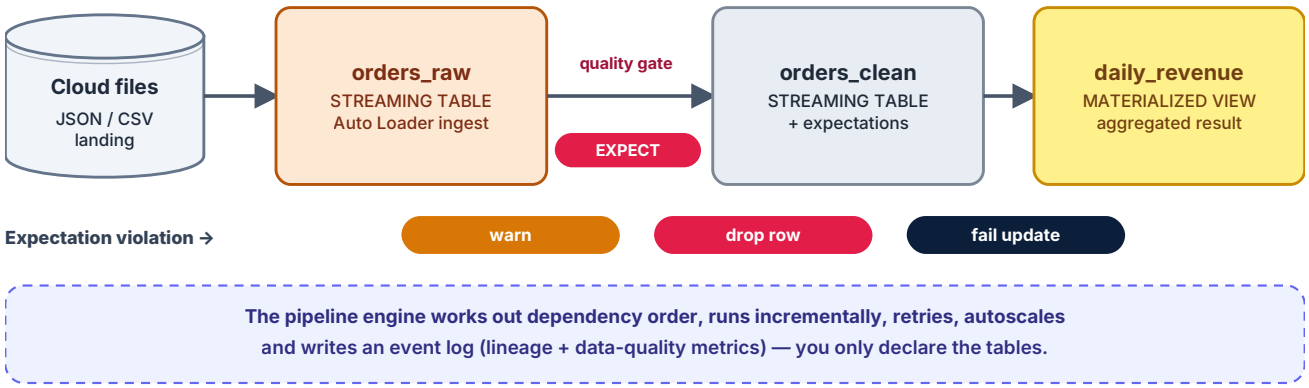


Figure 3.11 — A declarative pipeline with an expectation (quality gate) between Bronze and Silver

Object	What it is	Use for
Streaming table	Append-oriented table fed incrementally from a stream / file source	Ingestion (Bronze) and incremental Silver
Materialized view	Result of a query, kept up to date (incrementally where possible)	Aggregations and Gold tables
Temporary view	Intermediate result inside the pipeline only	Reusable logic, not published
Expectation	A data-quality rule: warn, drop row or fail the update	Enforce constraints without separate jobs

PYTHON · DECLARATIVE PIPELINE

```python
from pyspark import pipelines as dp          # (formerly: import dlt)
from pyspark.sql import functions as F

@dp.table                                        # streaming table
def orders_raw():
    return (spark.readStream.format('cloudFiles')
            .option('cloudFiles.format', 'json').load('/landing/orders'))

@dp.table
@dp.expect_or_drop('valid_id', 'order_id IS NOT NULL')
def orders_clean():
    return spark.readStream.table('orders_raw').withColumn('amount', F.col('amount').cast('double'))

@dp.materialized_view
def daily_revenue():
    return (spark.read.table('orders_clean')
            .groupBy(F.to_date('order_ts').alias('day')).agg(F.sum('amount').alias('revenue')))
```

SQL · SAME PIPELINE

```sql
CREATE OR REFRESH STREAMING TABLE orders_clean (
  CONSTRAINT valid_id EXPECT (order_id IS NOT NULL) ON VIOLATION DROP ROW
) AS SELECT * FROM STREAM(orders_raw);

CREATE OR REFRESH MATERIALIZED VIEW daily_revenue AS
SELECT to_date(order_ts) AS day, SUM(amount) AS revenue FROM orders_clean GROUP BY 1;
```

Why teams use it

- Less boilerplate: dependencies and ordering are automatic.
- Built-in data quality via expectations.
- Incremental processing and retries handled for you.
- Built-in CDC: `APPLY CHANGES INTO` (SCD1 / SCD2).

Compare

- **vs hand-written Spark jobs + Airflow**: declarative, less glue code.
- **vs dbt**: dbt is SQL-first and warehouse-agnostic; Lakeflow pipelines are Databricks-native with streaming built in.

3.12 dbt (Data Build Tool)

★★★

IN ONE LINE

SQL-first transformation framework: modular models, tests, docs and lineage that run inside your warehouse.

dbt (data build tool) is the **T in ELT**: you write `SELECT` statements as version-controlled **models**, and dbt compiles, orders and runs them inside your warehouse / lakehouse, adding **tests**, **documentation** and **lineage**. It brings software-engineering practice (Git, CI, modularity) to analytics SQL.

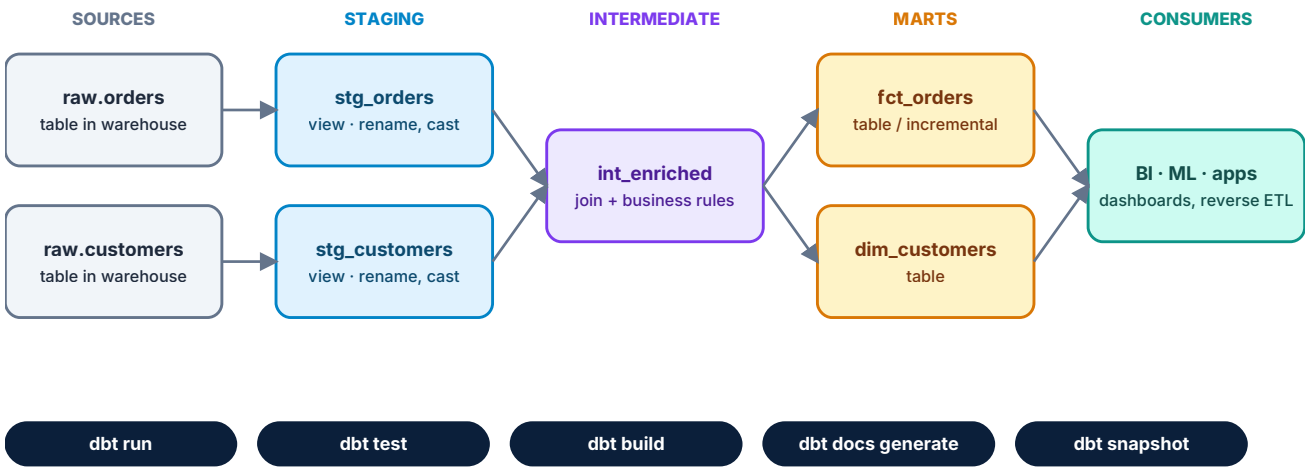


Figure 3.12 — A dbt project: sources → staging → intermediate → marts

DBT MODEL

```sql
-- models/marts/fct_orders.sql
{{ config(materialized='incremental',
unique_key='order_id') }}

select o.order_id, o.customer_id, o.amount,
o.updated_at
from {{ ref('int_orders_enriched') }} o
{% if is_incremental() %}
  where o.updated_at > (select max(updated_at)
from {{ this }})
{% endif %}
```

DBT TESTS

```yaml
# models/marts/schema.yml
models:
  - name: fct_orders
    columns:
      - name: order_id
        tests: [unique, not_null]
      - name: customer_id
        tests:
          - relationships:
              to: ref('dim_customers')
              field: customer_id
```

Materialization	What dbt creates	Use for
view	A SQL view	Light staging models (default)
table	Rebuilt table each run	Small/medium marts
incremental	Insert/merge only new or changed rows	Large facts
ephemeral	Inlined CTE, no object created	Reusable snippets
materialized_view	Warehouse-managed MV (where supported)	Auto-refreshed aggregates

Core concepts

- `ref()` / `source()` — build the dependency graph.
- **Tests**: generic (unique, not_null, accepted_values, relationships) and singular SQL tests.
- **Snapshots** — automatic SCD Type 2.
- **Macros** (Jinja), **seeds** (CSV), **docs** site with lineage.

What dbt is not

- Not an ingestion (E/L) tool.
- Not an orchestrator — it runs inside Airflow, Dagster, dbt Cloud or Workflows.
- It does not move data out of the warehouse; the engine does the compute.

3.13 CI/CD & DataOps

★★★

IN ONE LINE

Automating build, test and deployment of data pipelines — and running data like a reliable software product.

CI/CD applies software-delivery automation to data code: every change is built, tested and deployed through the same repeatable pipeline. **DataOps** is the broader practice that combines DevOps, agile and quality control to deliver reliable data *quickly and repeatedly*.

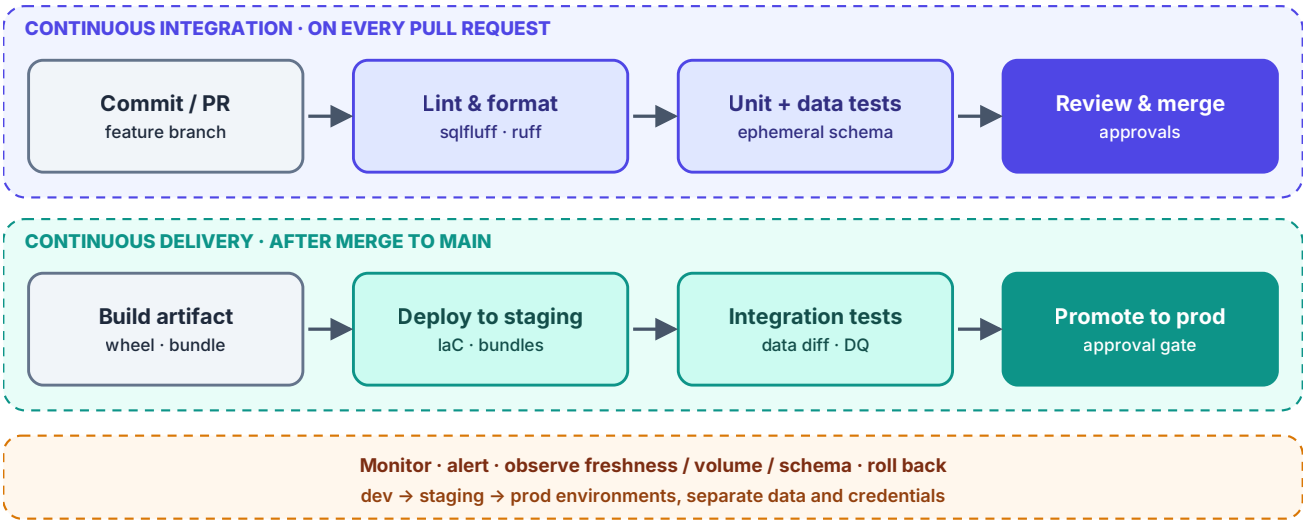


Figure 3.13 — A CI/CD pipeline for data (SQL, Spark, dbt, pipelines-as-code)

DataOps pillars

- **Version control** for code, configs, schemas, infrastructure.
- **Automated testing**: unit, integration, data-quality, regression.
- **Environments** (dev / staging / prod) with promotion gates.
- **Monitoring & observability** of freshness, volume, schema, cost.
- **Collaboration**, code review, shared definitions, data contracts.

Typical tools

- Git + GitHub Actions / GitLab CI / Azure DevOps / Jenkins
- Terraform, **Databricks Asset Bundles**, dbt Cloud / dbt slim CI (`state:modified+`)
- Great Expectations, Soda, dbt tests, Monte Carlo
- Airflow / Dagster / Workflows for orchestration

Test layer	Example	When
Static / lint	SQL style, type checks, secrets scan	Every commit
Unit	Transformation function on a tiny fixture DataFrame	Every PR
Data tests	unique / not_null / referential integrity on the built models	CI schema + after each prod run
Integration / regression	Run pipeline on a sample, compare output to the last release (data diff)	Before promotion
Monitoring	Freshness, volume anomalies, schema drift alerts	Continuously in prod

INTERVIEW ANGLE

Great interview detail: **test data, not just code**. Code can be correct while the data is wrong, so combine CI (code correctness) with runtime data-quality checks and observability. Also mention **idempotent deployments**, rollback plans and a CI schema per pull request so tests never touch production.

04

Part 4 · Operating at Scale & Interview Toolkit

The topics that separate good answers from great ones: performance, quality, orchestration, trade-offs, plus a design template, rapid-fire Q&A and glossary.

- Partitioning & small files
- Shuffle, skew & tuning
- Schema evolution & contracts
- Data quality & governance
- Orchestration (Airflow)
- Kafka · CAP · Sharding
- Data Vault & keys
- Design template · Q&A · Glossary

4.1 Partitioning, Clustering & the Small-File Problem

★★★

IN ONE LINE

Physically organising data so queries skip what they do not need.

Partitioning physically splits a table into folders by column values so queries read only the relevant slice (**partition pruning**). It is the cheapest big win in lake performance — and, done badly, the cheapest way to create a mess.

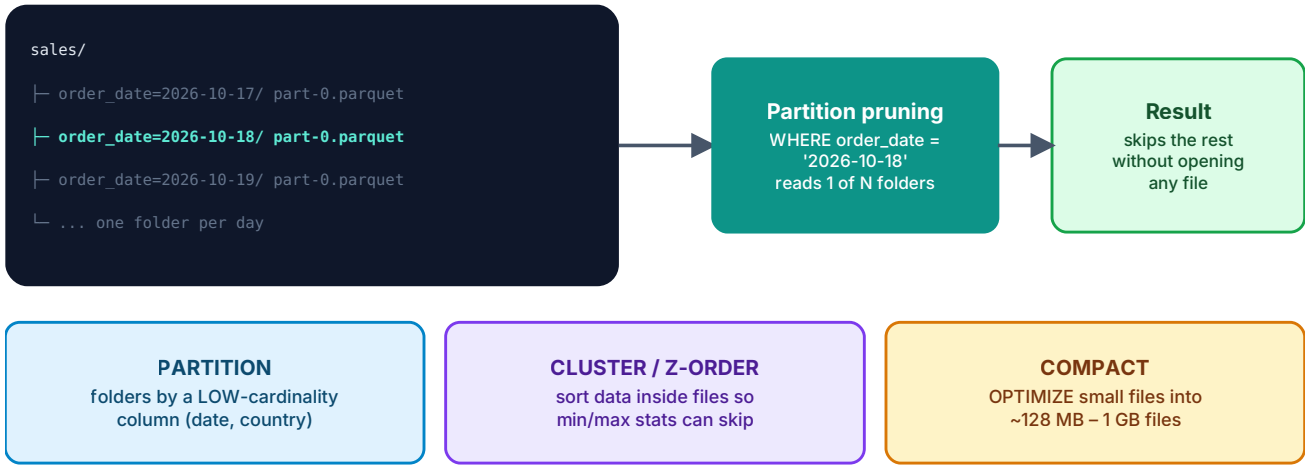


Figure 4.1 — Partition pruning, clustering and compaction

Concept	Rule of thumb
Partition column	Low cardinality, used in most filters (date is the classic). Aim for ≥ 1 GB of data per partition.
Over-partitioning	Partitioning by `user_id` or by hour on a small table creates millions of tiny files and slow metadata.
Small-file problem	Streaming and frequent micro-batches write many small files → slow listing, too many tasks. Fix with `OPTIMIZE` / auto-compaction.
Z-ORDER / liquid clustering	Co-locate rows with similar values in several columns (not just one) so min/max file statistics can skip files. Liquid clustering replaces static partitioning + Z-order and is changeable without rewriting.
Hidden partitioning	Iceberg derives partitions from column transforms (e.g. `days(ts)`) so queries do not need a separate partition column.

INTERVIEW ANGLE

Interview shortcut: **“partition by what you filter on, keep partitions big, cluster by high-cardinality filter columns, compact regularly.”**

4.2 Shuffle, Skew & Join Strategies

★★★

IN ONE LINE

Understanding data movement is the key to tuning distributed jobs.

Most slow Spark jobs come down to **shuffles**. A shuffle moves data across the network and disk so rows with the same key land together (joins, groupBy, distinct, orderBy). Reduce them, or make them cheaper.

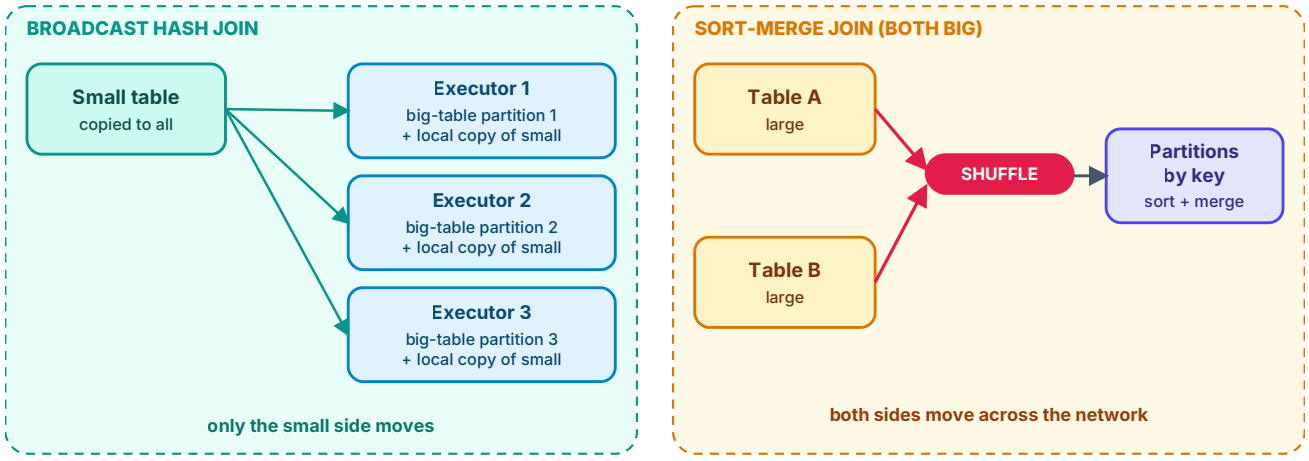


Figure 4.2a — Two join strategies: avoid shuffling the big side when you can

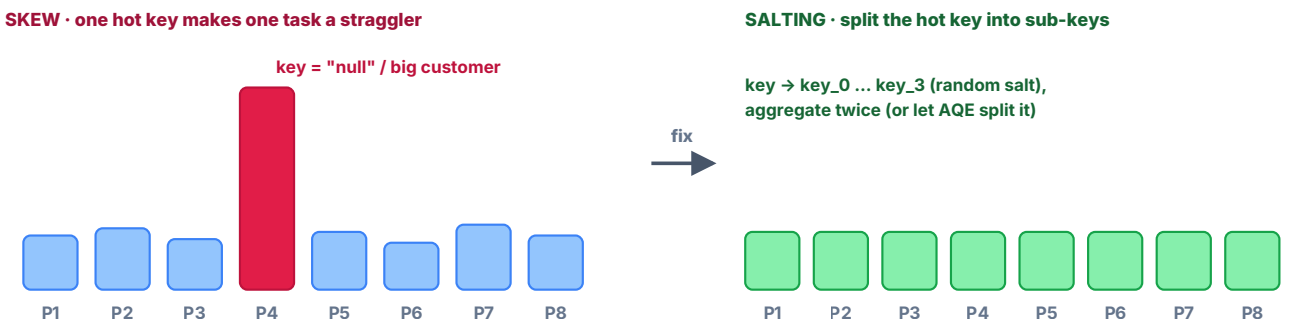


Figure 4.2b — Data skew and the salting fix

Problem	Symptom	Fixes
Shuffle-heavy join	Long stages, large shuffle read/write	Broadcast the small table (`broadcast(df)`, or auto-broadcast threshold); filter and select columns before the join
Data skew	Most tasks finish fast, one runs for ages	Enable **AQE skew-join handling**; salt the hot key; filter out null keys; isolate the hot key and broadcast it
Too few / too many partitions	OOM / spill, or thousands of tiny tasks	Tune `spark.sql.shuffle.partitions` (or rely on AQE coalescing); `repartition` before write
Spill to disk	Slow tasks, "spill" metrics	More memory per executor, fewer cores per executor, smaller partitions
Python UDFs	CPU-bound, serialisation overhead	Use built-in functions or pandas / Arrow UDFs

4.3 Schema Enforcement, Evolution & Data Contracts

★★☆

IN ONE LINE

Controlling how structure changes so downstream consumers do not break.

Source schemas change. A **schema-on-write** table **enforces** its schema (writes that do not match are rejected), while **schema evolution** lets you change it deliberately and safely. A **data contract** turns this into an agreement between producers and consumers.

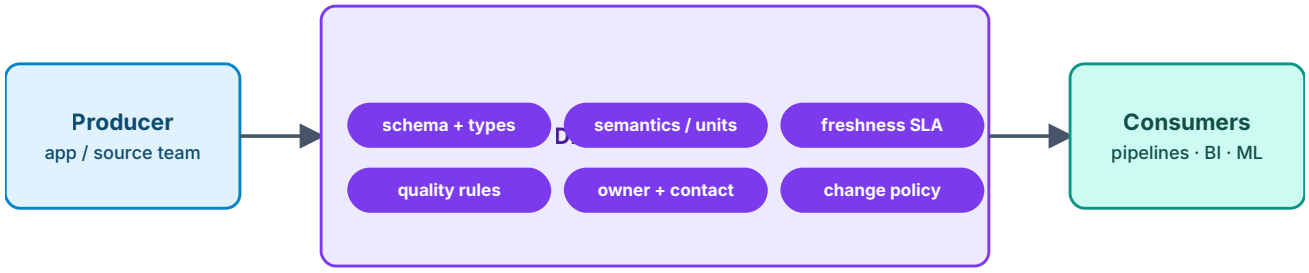


Figure 4.3 — A data contract sits between producer and consumers

Change	Backward compatible?	Typical handling
Add a nullable column	✓ yes	Schema evolution (e.g. Delta `mergeSchema`, Avro default values)
Widen a type (int → long)	✓ usually	Supported by many formats; verify downstream casts
Rename a column	× breaking	Column mapping (name-based), or add new + deprecate old
Drop a column	× breaking	Deprecate first, notify consumers, then remove
Change semantics / units	× silently breaking	New versioned field; contract review — the most dangerous kind

Schema enforcement vs evolution

Enforcement: reject bad writes (protects quality). **Evolution**: opt in to add columns (e.g. `.option("mergeSchema","true")`). Keep Bronze permissive (rescue unexpected columns) and Silver strict.

Tools

Schema registries for Kafka (Avro / Protobuf compatibility modes), dbt model contracts, Open Data Contract Standard, CI checks that fail PRs on breaking changes.

4.4 Data Quality, Observability, Lineage & Governance

★★★

IN ONE LINE

Making sure data is correct, fresh, traceable and appropriately accessed.

Pipelines that run successfully can still produce **wrong data**. Data quality checks, observability, lineage and governance are how teams find out before the business does.

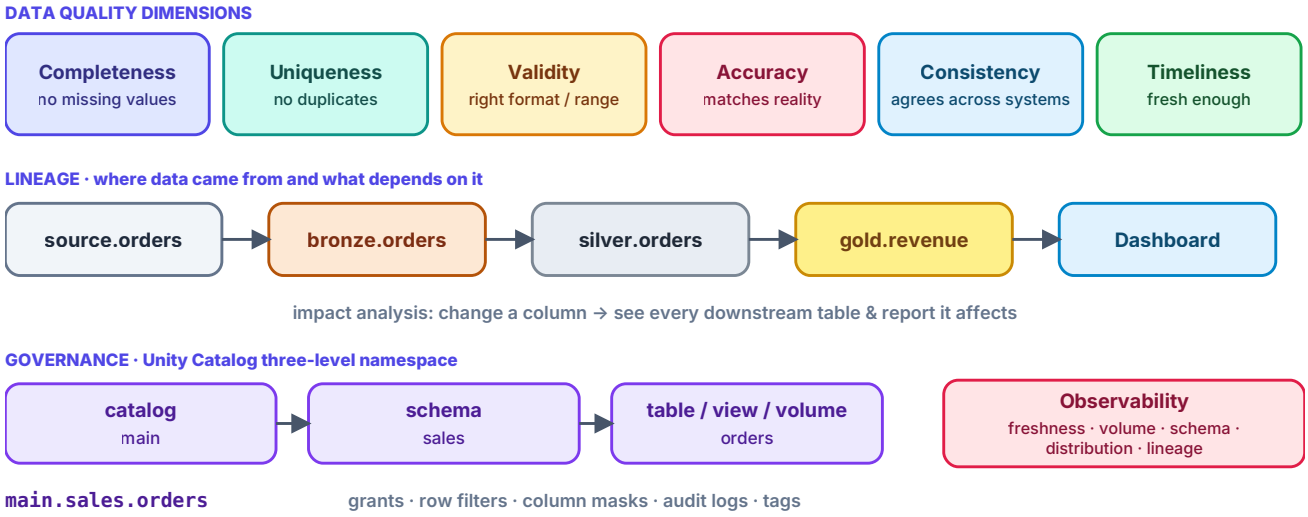


Figure 4.4 — Quality dimensions, lineage and catalog governance

Where to check

- **At ingestion**: schema, required fields, row counts vs source.
- **In transformation**: uniqueness, referential integrity, accepted values (dbt tests, DLT expectations).
- **After load**: reconciliation totals, freshness, anomaly detection.
- Route failures to a **quarantine** table, not the void.

Observability signals

- **Freshness** — did data arrive on time?
- **Volume** — unexpected row-count drop/spike.
- **Schema** — drift or breaking change.
- **Distribution** — null rate, value ranges shift.
- **Lineage** — who is impacted.

Tool	Role
dbt tests, DLT expectations, Great Expectations, Soda	Declarative data-quality rules in the pipeline
Unity Catalog, Glue Catalog, DataHub, Collibra, Alation	Catalog, governance, access control, lineage
Monte Carlo, Databand, Elementary	Data observability and anomaly detection

INTERVIEW ANGLE

A strong answer includes **what you do when a check fails**: fail fast for critical constraints, quarantine for row-level issues, alert the owner, and never silently publish bad data to Gold.

4.5 Orchestration (Airflow & DAGs)

★★★

IN ONE LINE

Scheduling, ordering, retrying and monitoring the tasks that make up pipelines.

An orchestrator schedules and coordinates tasks: it runs them in dependency order, retries failures, tracks state, supports backfills and alerts people. **Apache Airflow** is the best-known; alternatives include Dagster, Prefect, Databricks Workflows, Azure Data Factory and AWS Step Functions.

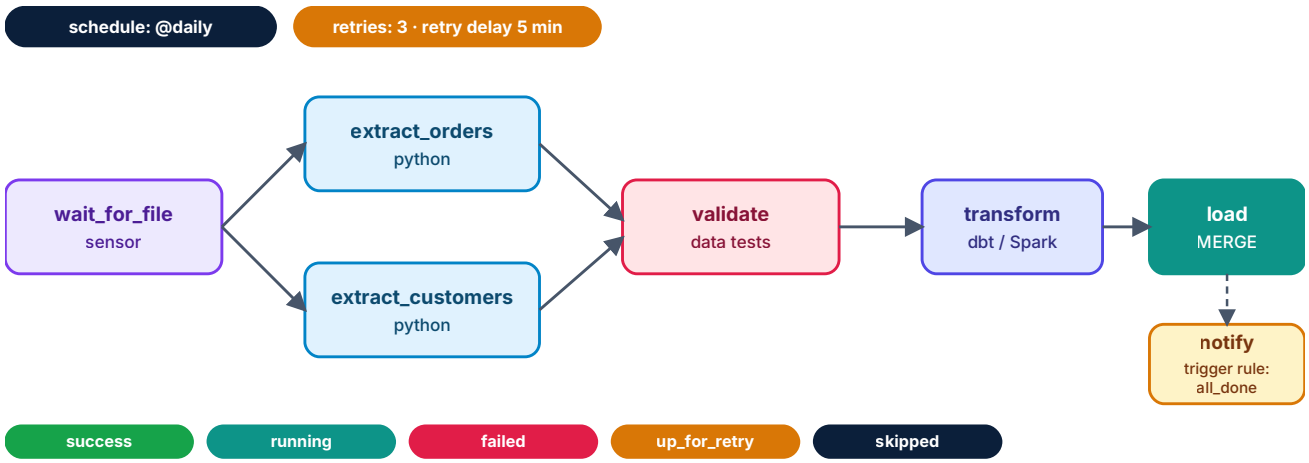


Figure 4.5 — An Airflow-style DAG: tasks, dependencies, states

Airflow vocabulary

- **DAG** — directed acyclic graph of tasks (no cycles).
- **Operator** / **task** — a unit of work; **sensor** waits for a condition.
- **Scheduler**, **executor**, **workers**, metadata DB.
- **XCom** — tiny messages between tasks (not big data!).
- **Catchup / backfill** — run past schedule intervals.

Good practices

- Make tasks **idempotent** and parameterised by logical date.
- Orchestrate, do not compute: push heavy work to Spark / the warehouse.
- Set retries, timeouts, SLAs and failure alerts.
- Keep DAG files light; no heavy code at import time.
- Orchestration (central scheduler) vs choreography (event-driven services).

4.6 Kafka & Message Brokers

★★★

IN ONE LINE

A durable, replayable event log that decouples producers from consumers.

Apache Kafka is a **distributed, durable, append-only commit log** used as the backbone for event streaming. Producers write events to **topics**; consumers read them at their own pace, and events are retained (days to forever) so they can be replayed.

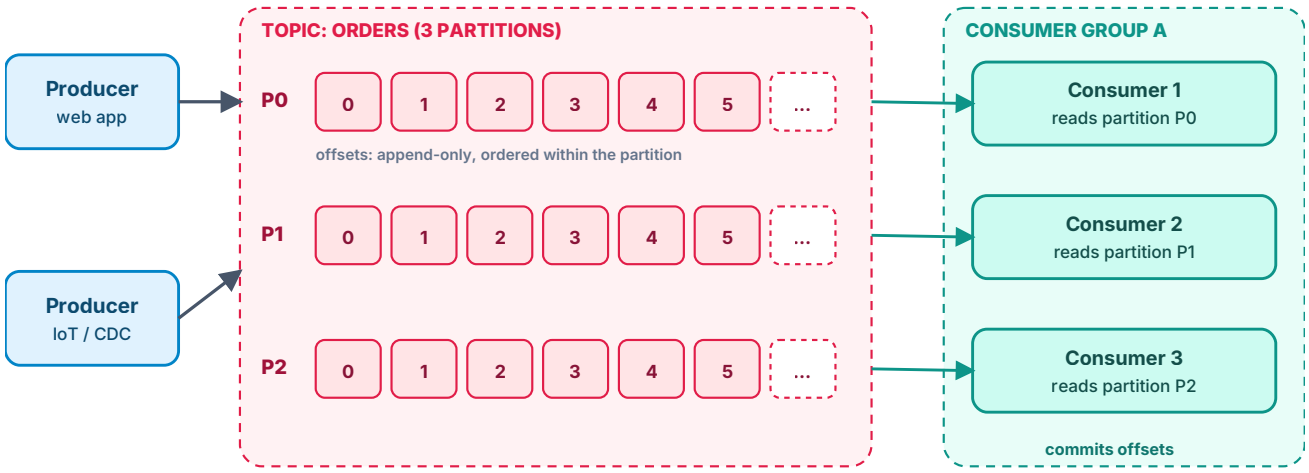


Figure 4.6 — Topics, partitions, offsets and consumer groups

Concept	Meaning
Topic / partition	A topic is split into partitions for parallelism; **ordering is guaranteed only within a partition**.
Key	Events with the same key go to the same partition (e.g. keep all events of one customer in order).
Offset	Position of a record in a partition; consumers track and commit it, enabling replay and restart.
Consumer group	Consumers share partitions (max 1 consumer per partition in a group). Different groups read independently.
Replication / ISR	Each partition has a leader and followers; acks=all with in-sync replicas gives durability.
Retention / compaction	Delete by age/size, or *compact* to keep the latest value per key (changelog topics).

INTERVIEW ANGLE

Kafka vs a queue (RabbitMQ/SQS): Kafka is a **log** — messages are not removed when read, multiple groups can consume, and you can replay. This ties directly to Kappa architecture and CDC.

4.7 CAP Theorem, Replication & Sharding

★★☆

IN ONE LINE

Fundamental trade-offs when data is spread across machines.

The **CAP theorem**: during a network **P**artition, a distributed data store must choose between **C**onsistency (every read sees the latest write) and **A**vailability (every request gets a response). Since partitions are unavoidable in distributed systems, the real choice is **CP vs AP**.

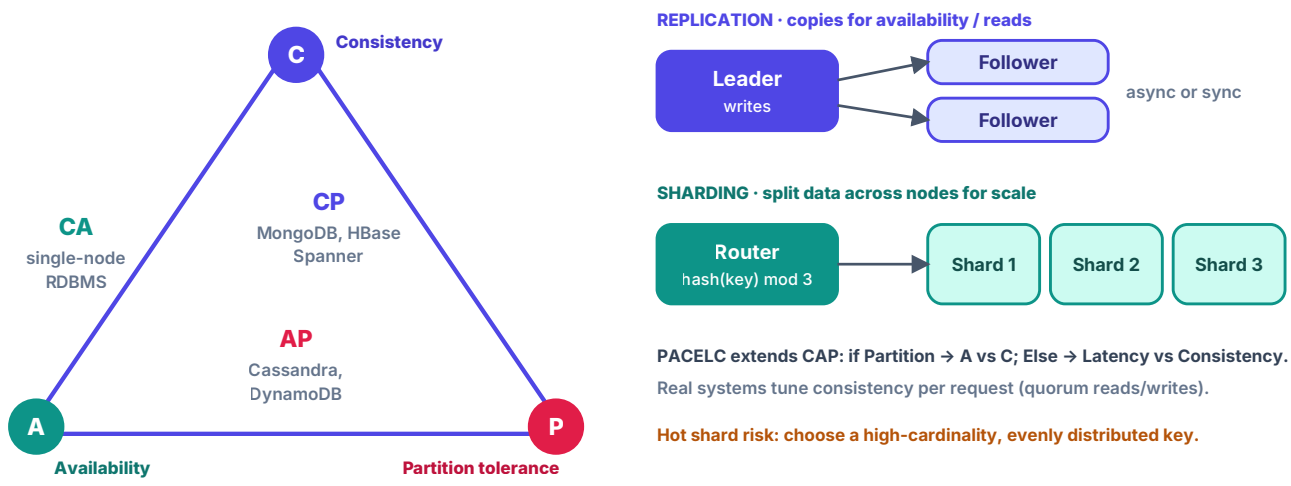


Figure 4.7 — CAP, replication and sharding

Term	Meaning	Why interviewers ask
Replication	Keep copies of data on multiple nodes (leader–follower or multi-leader)	Availability, read scaling, durability, lag / consistency trade-offs
Sharding / partitioning	Split data horizontally across nodes by key	Write scaling; hot keys; re-sharding cost
Eventual consistency	Replicas converge if writes stop	BASE systems, DynamoDB / Cassandra reads
Quorum	R + W > N guarantees overlap of read and write sets	Tunable consistency

REMEMBER

Remember the mix-up from 1.2: **C in CAP** is about replicas agreeing; **C in ACID** is about data satisfying integrity rules.

4.8 Data Vault, Surrogate Keys & Modeling Choices

★★☆

IN ONE LINE

An audit-friendly integration model, plus the key-design ideas behind every warehouse.

Data Vault separates **business keys** (Hubs), **relationships** (Links) and **descriptive history** (Satellites). It is built for auditability, parallel loading and absorbing source changes without redesign. It usually feeds dimensional marts on top.

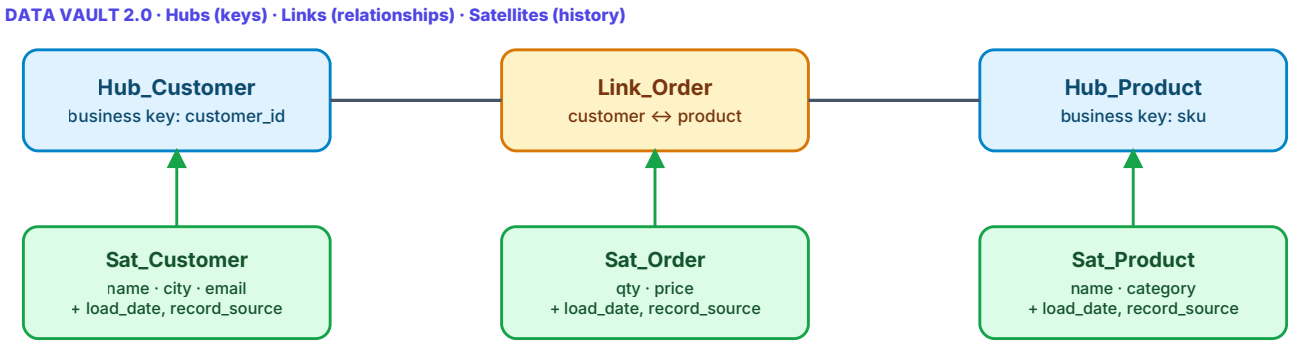


Figure 4.8 — Hubs, Links and Satellites

Surrogate vs natural keys

- **Natural / business key**: meaningful, from the source (email, SKU) — can change or collide across systems.
- **Surrogate key**: system-generated integer or hash with no meaning — stable, joins fast, insulates the warehouse from source changes.
- Hash keys (MD5 / SHA of business key) allow parallel, order-independent loads.

When to use what

- **Star schema**: BI-facing, simplicity and speed.
- **Data Vault**: large enterprises with many changing sources, strict audit needs.
- **One Big Table**: ML features and single-purpose dashboards.

4.9 Performance Tuning Cheat Sheet

★★★

IN ONE LINE

A symptom → cause → fix map for slow and expensive pipelines.

A compact troubleshooting map. Interviewers love "this job is slow — what do you check?". Start from the **Spark UI / query profile** and work through the symptoms.

Symptom	Likely cause	First fixes
Reading far more data than needed	No pruning; `SELECT *`	Select only needed columns; filter on partition / clustered columns; use columnar formats
Thousands of tiny tasks / slow listing	Small-file problem	`OPTIMIZE`, auto-compaction, larger micro-batch interval, fewer partitions
One task runs far longer than the rest	Data skew	AQE skew join, salting, filter null keys
Huge shuffle read / write	Wide transformations on big data	Broadcast small tables, pre-aggregate, filter early, reduce columns
Out-of-memory / spill	Large partitions, big collect, wide rows	More partitions, bigger executors, avoid `collect()` / `toPandas()` on big data
Slow MERGE	Scans whole target	Add partition / cluster predicates to `ON`, dedupe source, compact target
Slow Python code	Row-wise UDFs	Built-in functions, pandas / Arrow UDFs, push logic to SQL
Costs keep rising	Over-provisioned or always-on clusters	Auto-scaling, spot / serverless, auto-terminate, `availableNow` triggers, cache only what is reused

INTERVIEW ANGLE

Structure your answer: **measure → locate the bottleneck stage → identify cause (IO, shuffle, skew, spill, small files) → apply the targeted fix → re-measure**. Never start by "add more nodes".

4.10 How to Answer a Pipeline Design Question

★★★

IN ONE LINE

A repeatable eight-step framework for system-design style interview questions.

System-design questions ("design a pipeline for X") reward a **repeatable structure**. Walk through these eight steps out loud, and tie each to the concepts in this handbook.

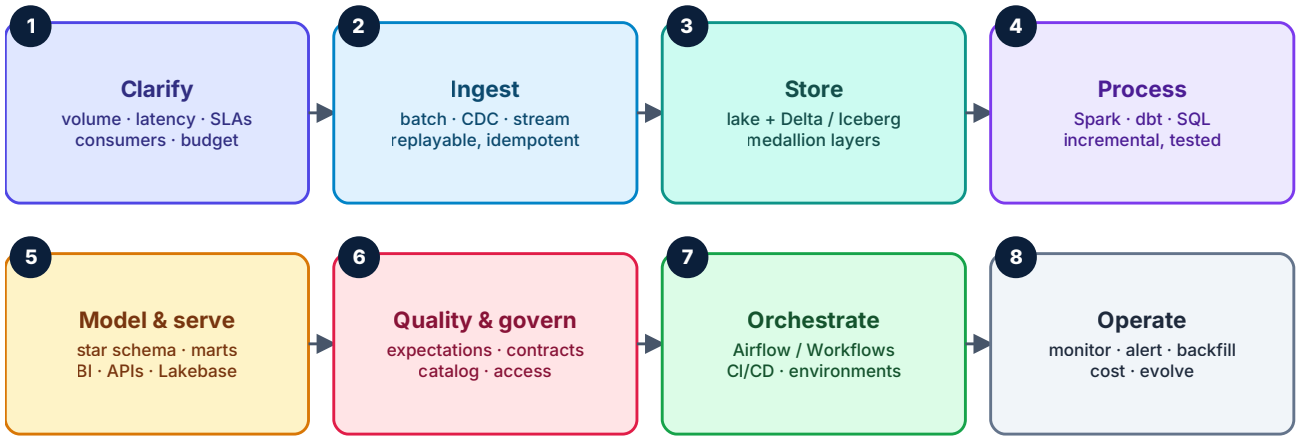


Figure 4.9 — An 8-step template for data-pipeline design answers

Step	Questions to ask / things to mention
1 Clarify	Data volume and growth? Freshness (latency) requirement? Consumers? Accuracy vs speed? Compliance (PII)? Budget?
2 Ingest	Full vs **incremental**; **CDC** for databases; Kafka for events; **idempotent** and replayable; handle late data (**watermarks**, lookback).
3 Store	Lakehouse with **Delta / Iceberg**, **Parquet**, **medallion** layers, sensible partitioning / clustering.
4 Process	Spark or dbt/SQL; **MERGE** for upserts; SCD2 where history matters; avoid shuffles and skew.
5 Model & serve	Star schema in Gold; marts per domain; BI via SQL warehouse; low-latency serving store (e.g. Lakebase / Postgres) if apps need it.
6 Quality & governance	Expectations / dbt tests, contracts, quarantine, lineage, catalog, row/column security.
7 Orchestrate & ship	DAG with retries, backfills; CI/CD with dev/stage/prod; infrastructure as code.
8 Operate	Freshness / volume alerts, SLAs, runbooks, cost control, schema evolution plan.

INTERVIEW ANGLE

Close with **trade-offs and alternatives** ("if latency were seconds I would add Kafka + streaming; if volume were small I would drop Spark for plain SQL"). Showing you can *change* the design when requirements change is what interviewers score highest.

4.11 Rapid-Fire Interview Q&A

★★★

IN ONE LINE

Twenty questions that cover most of the handbook, with answers short enough to say out loud.

Q What is the difference between a data lake, warehouse and lakehouse?

Warehouse: structured, schema-on-write, fast BI, higher cost. Lake: cheap storage of any data, schema-on-read, no ACID. Lakehouse: lake storage + open table format (Delta/Iceberg) giving ACID, governance and BI performance on one copy of data.

Q How does a WAL guarantee durability?

Changes are appended to a sequential log and fsynced at COMMIT before the client is acknowledged. Data pages are flushed later; after a crash the DB replays the WAL (REDO) and rolls back uncommitted work.

Q How does Delta Lake provide ACID on S3?

Data files are immutable Parquet; the _delta_log records add/remove actions as numbered commits. A commit is the atomic creation of the next log file (optimistic concurrency). Readers only trust files in the committed snapshot.

Q Parquet vs Delta?

Parquet is a columnar file format. Delta is a table format: Parquet files + a transaction log giving ACID, time travel, schema enforcement and MERGE.

Q Star vs snowflake schema?

Star: denormalised dimensions, fewer joins, faster and simpler. Snowflake: normalised dimensions, less redundancy, more joins. Star is the default for analytics.

Q Explain SCD Type 2.

Keep history by expiring the old dimension row (valid_to, is_current=false) and inserting a new row with a new surrogate key. Facts join to the version that was current at event time.

Q ETL vs ELT?

ETL transforms before loading in a separate engine; ELT loads raw data first and transforms inside the warehouse/lakehouse, keeping raw data for reprocessing.

Q What is idempotency and how do you achieve it?

Re-running gives the same result. Use MERGE on stable keys, partition overwrite, dedupe, processed-file tracking and deterministic transforms.

Q Can you get exactly-once?

Effectively yes: at-least-once delivery plus idempotent or transactional sinks and checkpointed offsets (Kafka transactions, Spark checkpoints + Delta). External non-idempotent systems need your own idempotency keys.

Q Two meanings of watermark?

Batch: stored last-loaded timestamp/ID for incremental loads. Streaming: event-time threshold (max event time minus allowed lateness) that decides when windows close and late data is dropped.

Q How do you handle late-arriving data?

Streaming watermarks with allowed lateness; batch lookback window reprocessing the last N days with MERGE/ partition overwrite; placeholder members for late dimensions.

Q Why is CDC better than polling updated_at?

Log-based CDC captures deletes and every intermediate change, preserves ordering and adds almost no load on the source. Polling misses hard deletes and can lose rows at boundaries.

Q Lambda vs Kappa?

Lambda: separate batch and speed layers merged at serving (two codebases). Kappa: one streaming pipeline on a replayable log; reprocess by replay. Lakehouse tables often unify both.

Q What is the small-file problem?

Many tiny files cause slow listing, high metadata overhead and too many tasks. Fix with compaction (OPTIMIZE), better partitioning, larger batch intervals and auto-compaction.

Q Narrow vs wide transformations?

Narrow: no data movement (filter, map). Wide: shuffle across the network (groupBy, join, distinct) and define stage boundaries.

Q How do you fix data skew?

AQE skew-join handling, salting hot keys, broadcast the small side, filter null/default keys, better partition keys.

Q What does the medallion architecture buy you?

Progressive quality layers: Bronze raw and replayable, Silver cleaned and conformed, Gold business-ready. Easier reprocessing, clearer contracts, tiered access.

Q Data mesh vs data fabric?

Mesh is an organisational model (domain ownership, data products, federated governance). Fabric is a technology approach (active metadata + automation integrating distributed data). Complementary.

Q What is Lakebase?

Databricks' managed serverless Postgres OLTP database integrated with the lakehouse (Neon-based), synced with Delta tables so apps and analytics share governed data.

Q How would you test a data pipeline?

Unit tests on transforms, data tests (unique, not_null, relationships), CI in an isolated schema, regression / data-diff before promotion, freshness and volume monitoring in production.

4.12 Glossary: Every Term in One Line

★★☆

IN ONE LINE

A quick-revision index of all the concepts in this handbook.

Every term from your list, plus the extras, in one place. The last column points to the section with the full explanation and diagram.

Term	One-line definition	§
ACID	Atomicity, Consistency, Isolation, Durability: transaction guarantees.	1.2
Atomicity	A transaction fully commits or has no effect.	1.3
Back-date refresh	Re-processing past periods for late data, fixes or new logic.	3.4
Batch processing	Processing bounded data on a schedule.	3.8
Bronze / Silver / Gold	Raw, cleaned and business-ready lakehouse layers.	2.12
CDC	Capturing row-level inserts, updates, deletes from a source.	3.10
CI/CD	Automated build, test and deployment of code.	3.13
CRUD	Create, Read, Update, Delete.	1.1
Data contract	Producer–consumer agreement on schema, semantics and SLAs.	4.3
Data fabric	Metadata-driven automated integration layer over distributed data.	2.11
Data lake	Cheap object storage for raw data of any type; schema-on-read.	2.2
Data lakehouse	Lake storage + table format giving warehouse reliability and speed.	2.3
Data mart	Subject-specific slice of analytical data.	2.9
Data mesh	Decentralised, domain-owned data products on a self-serve platform.	2.10
Data modeling	Designing data structures from concept to physical tables.	1.9
Data Ops	DevOps-style automation, testing and monitoring for data.	3.13
Data warehouse	Modelled, integrated, historical store for analytics.	2.1
dbt	SQL transformation framework with tests, docs and lineage.	3.12
Delta Lake	Open table format: Parquet + transaction log.	2.5
Delta Live Tables	Declarative pipelines; now Lakeflow Spark Declarative Pipelines.	3.11
Dimensional modeling	Fact and dimension tables around a business-process grain.	1.10
ELT	Extract, Load, then transform inside the platform.	3.1
ETL	Extract, Transform, then load.	3.1
Exactly-once	Each record's effect applied once despite retries.	3.6
Idempotency	Repeated runs give the same result.	3.5
Incremental loading	Loading only new or changed data.	3.2
Kafka	Distributed, replayable event log.	4.6
Kappa architecture	Single streaming pipeline over a replayable log.	3.9

Term	One-line definition	§
Lakebase	Databricks serverless Postgres OLTP integrated with the lakehouse.	2.8
Lambda architecture	Parallel batch and speed layers merged at serving.	3.9
Medallion architecture	Bronze → Silver → Gold refinement layers.	2.12
Normalization	Removing redundancy by splitting tables (1NF–3NF).	1.8
OLAP	Analytical, read-heavy, large-scan workloads.	1.7
OLTP	Transactional, many small concurrent operations.	1.7
Open table format	Metadata layer giving ACID tables on files (Delta, Iceberg, Hudi).	2.5
Parquet	Columnar, compressed file format for analytics.	2.4
Partitioning	Splitting data into folders for pruning.	4.1
Pipeline	Automated flow moving and transforming data.	3.1
Snowflake schema	Star schema with normalised dimensions.	1.10
Spark	Distributed in-memory data processing engine.	2.6
SCD	Slowly Changing Dimensions: managing attribute history.	1.11
Star schema	Central fact table with denormalised dimensions.	1.10
Streaming processing	Processing unbounded events continuously.	3.8
Transaction log	Ordered record of table changes (e.g. _delta_log).	2.5
Trino	Distributed MPP SQL engine for federated queries.	2.7
Upsert	Update if exists, otherwise insert (MERGE).	3.7
WAL	Write-Ahead Log: log changes before data files.	1.4
WAL buffer	In-memory staging area for WAL records before flush.	1.4
Watermark	Last loaded value (batch) or event-time progress marker (streaming).	3.3